

LINUX INTERGALÁCTICO: BASE DA
JORNADA JEDI

ADMINISTRAÇÃO DO PINGUIM



4

LISTA DE FIGURAS

Figura 1 – Exemplo de execução do comando ip link show	6
Figura 2 – Exemplo de execução do comando ip link show com parâmetro dev	7
Figura 3 – Exemplo de execução do comando ip addr show	8
Figura 4 – Exemplo de execução do comando ping	8
Figura 5 – Exemplo de execução do comando ping com parâmetro -c	9
Figura 6 – Exemplo de execução do comando ip route show	9
Figura 7 – Exemplo de execução de remoção e adição de endereço IPv4	10
Figura 8 – Exemplo de execução de endereço IPv4 modificado	10
Figura 9 – Testes de conectividade com o comando ping	10
Figura 10 – Exemplo de execução e teste de adição de gateway padrão	11
Figura 11 – Exemplo de execução do comando nmap	12
Figura 12 – Exemplo de execução do comando nmap com análise aprofundada	12
Figura 13 – Exemplo de execução do comando ps	14
Figura 14 – In your hands, zombie	15
Figura 15 – Exemplo de execução do top	16
Figura 16 – Exemplo de execução do comando ps aux	17
Figura 17 – Exemplo de execução do comando ps aux grep gnome-chess	17
Figura 18 – Exemplo de execução do comando kill	18
Figura 19 – Comando killall em ação	19
Figura 20 – Exemplo de execução do comando killall	19
Figura 21 – Melhor exemplo da sutileza (ao não deixar vestígios) do killall	20
Figura 22 – Exemplo de execução do comando GNOME Chess travando o terminal de comandos	21
Figura 23 – Exemplo de execução do comando GNOME Chess rodando em <i>background</i>	21
Figura 24 – Exemplo de execução do comando screen -ls	22
Figura 25 – Criação simples de um novo arquivo e sua listagem	23
Figura 26 – Mudando o dono do arquivo com chown	25
Figura 27 – Mudando o dono do arquivo e grupo dono com chown	25
Figura 28 – Mudando o grupo dono do arquivo com chgrp	26
Figura 29 – Mudando as permissões de grupo com o comando chmod	26
Figura 30 – Mudando as permissões gerais com o comando chmod	27
Figura 31 – O pinguim remoto	28
Figura 32 – Um Raspberry Pi 3 model B, um pequeno notável	29
Figura 33 – Exemplo de execução do comando SSH	30
Figura 34 – Exemplo de execução do comando SSH ao sair do <i>host</i> remoto	30
Figura 35 – Adicionando um host no arquivo /etc/hosts	31
Figura 36 – Exemplo do comando SSH utilizando <i>hostname</i>	32
Figura 37 – Exemplo do comando scp transferindo um arquivo local para um servidor remoto	33
Figura 38 – Exemplo do comando scp transferindo um arquivo de um servidor remoto para o local	33
Figura 39 – Instalação do serviço de ssh	34
Figura 40 – Subindo o serviço de SSH	34
Figura 41 – Tela inicial do vi criando arquivo.txt	37
Figura 42 – Editando um arquivo com vi usando [i]	38
Figura 43 – Editando um arquivo com vi usando [a]	38

Figura 44 – Editando um arquivo com vi usando [A]	38
Figura 45 – Editando um arquivo com vi usando [o]	39
Figura 46 – Exemplo de documento no vi	40
Figura 47 – Exemplo de documento no vi com numeração de linhas	41
Figura 48 – Exemplo para acessar uma linha específica no vi.....	41
Figura 49 – Exemplo de como copiar e colar uma linha no vi	42
Figura 50 – Exemplo de como copiar um trecho do arquivo para outro	43
Figura 51 – Exemplo de busca por palavra-chave no vi.....	44
Figura 52 – Exemplo de substituição de termos no vi	45



LISTA DE LINHAS DE COMANDO

Comando de prompt 1 – Verificando as conexões de rede.....	6
Comando de prompt 2 – Comando para remoção e adição de endereço IPv4 na interface de rede	9
Comando de prompt 3 – Comando para adição de gateway padrão	11
Comando de prompt 4 – Exemplo do comando kill congelando o processo de PID 2829	18
Comando de prompt 5 – Dando ao usuário “fiap” a posse de todos os arquivos e subdiretórios.....	25
Comando de prompt 6 – Habilitando no boot e iniciando imediatamente o serviço de ssh.....	34
Comando de prompt 7 – Exemplo do comando vi criando ou editando arquivo.txt...	36
Comando de prompt 8 – Exemplo de substituição da palavra “mais” por “menos” no vi.....	45

SUMÁRIO

ADMINISTRAÇÃO DO PINGUIM.....	6
1. INTRODUÇÃO	6
2 CONFIGURAÇÃO DE REDE	6
2.1 Comandos ip e ping	6
2.2 nmap – Escaneamento de portas.....	11
3 GERENCIAMENTO DE PROCESSOS	13
3.1 PS – listar processos.....	13
3.2 Top – monitor de processos	14
3.3 Kill e Killall – matando processos	16
3.4 & – Rodando processos em background.....	20
3.5 Screen – criando terminais virtuais	22
4 PERMISSÕES: PODE OU NÃO PODE?	23
4.1 chown	25
4.2 chgrp	26
4.3 chmod.....	26
5 ACESSO A MÁQUINAS REMOTAS COM SSH.....	27
5.1 SSH – acesso a outra máquina remotamente	28
5.2 SCP – realizando transferências de arquivo seguras.....	32
5.3 Subindo um serviço SSH.....	33
5.4 Tornando o SSH ainda mais seguro	35
6 VI: O MELHOR EDITOR DE TEXTO	36
6.1 Entrando e saindo do vi.....	36
6.2 Os modos de comando e de edição de texto	37
6.3 Salvando o arquivo.....	39
6.4 Numeração de linhas e como ir direto para uma delas	39
6.5 Copiando e colando textos	42
6.6 Apagando textos	43
6.7 Localizando e substituindo palavras.....	44
CONCLUSÃO.....	45
REFERÊNCIA	47
GLOSSÁRIO	48

ADMINISTRAÇÃO DO PINGUIM

1. INTRODUÇÃO

Retomaremos nosso aprendizado dos comandos essenciais e abordaremos, neste capítulo, o gerenciamento de rede, processos, permissionamento de arquivos e pastas, possibilidade de acesso remoto com SSH, cópias seguras com o SCP e o uso de um dos melhores processadores de texto em modo texto, o famoso VI.

2 CONFIGURAÇÃO DE REDE

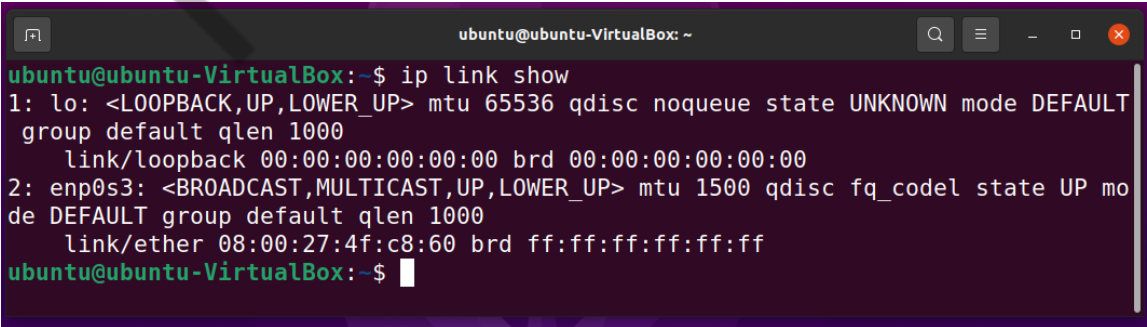
Saber como configurar as interfaces de rede em um sistema operacional Linux possibilitará a comunicação desta máquina com outras em uma rede local ou na Internet, logo, os comandos são essenciais, como veremos a seguir.

2.1 Comandos ip e ping

O comando ifconfig permite a configuração das informações mais básicas de uma interface de rede, como seu IP e a máscara de rede.

```
$ ip link show
```

Comando de prompt 1 – Verificando as conexões de rede
Fonte: Elaborado pelo autor (2021)

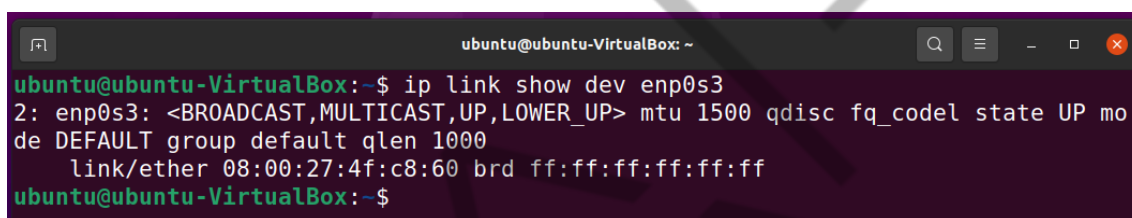


```
ubuntu@ubuntu-VirtualBox: ~  
ubuntu@ubuntu-VirtualBox:~$ ip link show  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT  
    group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mo  
de DEFAULT group default qlen 1000  
    link/ether 08:00:27:4f:c8:60 brd ff:ff:ff:ff:ff:ff  
ubuntu@ubuntu-VirtualBox:~$
```

Figura 1 – Exemplo de execução do comando ip link show
Fonte: Elaborado pelo autor (2021)

A interface `lo` é o loopback, sempre presente em qualquer sistema operacional. Seu ip padrão `127.0.0.1` indica como posso apontar para minha própria máquina, mesmo que a interface de rede real esteja inativa.

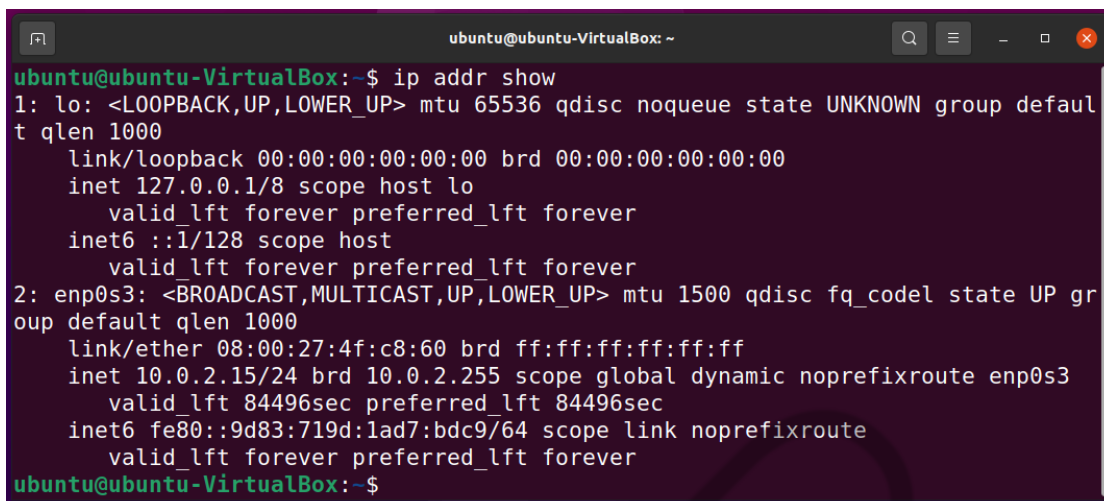
Se por acaso o equipamento possuir muitas interfaces de rede, há a possibilidade de filtrar melhor as informações ali dispostas, trazendo apenas a interface desejada. Para isso, o comando `ip` possui um parâmetro `dev` (de *device* ou dispositivo) em que é possível dizer de qual deles queremos realmente ver as informações. O comando `ip link show dev enp0s3`, por exemplo, traz apenas a interface de rede `enp0s3`, removendo completamente qualquer menção da interface `lo` (loopback), mostrada na execução anterior.



```
ubuntu@ubuntu-VirtualBox: ~  
ubuntu@ubuntu-VirtualBox:~$ ip link show dev enp0s3  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mo  
de DEFAULT group default qlen 1000  
    link/ether 08:00:27:4f:c8:60 brd ff:ff:ff:ff:ff:ff  
ubuntu@ubuntu-VirtualBox:~$
```

Figura 2 – Exemplo de execução do comando `ip link show` com parâmetro `dev`
Fonte: Elaborado pelo autor (2021)

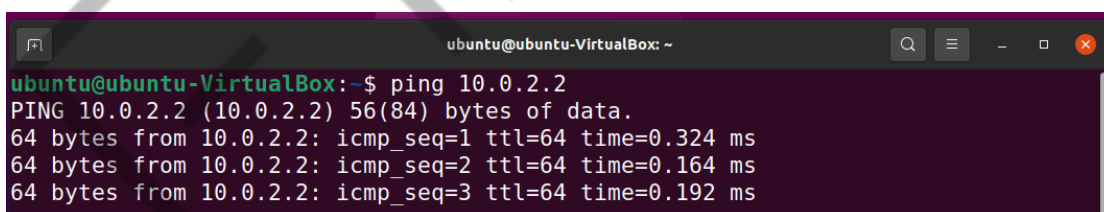
Se a ideia é visualizar os endereços IP das interfaces de rede, sejam estes endereços IPv4 ou IPv6, o comando a ser utilizado é o `ip addr show`, como pode ser visto a seguir: repare que a interface `lo` (loopback) possui seu clássico endereçamento IPv4 `127.0.0.1`, enquanto podemos ver a interface descrita neste Ubuntu como `enp0s3` com um endereçamento IPv4 de `10.0.2.15` e um comprimento de prefixo `/24`, que indica uma máscara de sub-rede `255.255.255.0`, ou seja, uma sub-rede com 253 posições de hosts disponíveis.



```
ubuntu@ubuntu-VirtualBox: ~  
ubuntu@ubuntu-VirtualBox:~$ ip addr show  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host  
        valid_lft forever preferred_lft forever  
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:4f:c8:60 brd ff:ff:ff:ff:ff:ff  
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3  
        valid_lft 84496sec preferred_lft 84496sec  
    inet6 fe80::9d83:719d:1ad7:bdc9/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
ubuntu@ubuntu-VirtualBox:~$
```

Figura 3 – Exemplo de execução do comando `ip addr show`
Fonte: Elaborado pelo autor (2021)

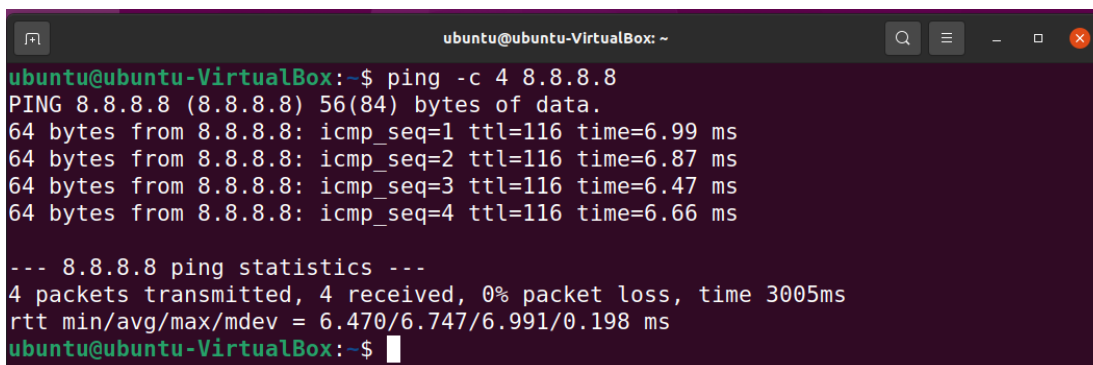
Vamos aproveitar este momento para testar a conectividade de rede desta interface. Há outro nesta rede no IPv4 10.0.2.2, representando, provavelmente, a máquina hospedeira. Ao usar o comando `ping`, é possível enviar um pacote de rede do tipo echo request e, se a conectividade estiver boa (e o outro host estiver autorizado em seu firewall a responder, é claro), há, em troca, um echo reply. Basta digitar o comando `ping` e o ip de destino para se obter diversas métricas interessantes a respeito da conexão entre estes dois hosts.



```
ubuntu@ubuntu-VirtualBox: ~  
ubuntu@ubuntu-VirtualBox:~$ ping 10.0.2.2  
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data.  
64 bytes from 10.0.2.2: icmp_seq=1 ttl=64 time=0.324 ms  
64 bytes from 10.0.2.2: icmp_seq=2 ttl=64 time=0.164 ms  
64 bytes from 10.0.2.2: icmp_seq=3 ttl=64 time=0.192 ms
```

Figura 4 – Exemplo de execução do comando `ping`
Fonte: Elaborado pelo autor (2021)

Aproveitamos aqui também para testar a conectividade externa, ao pingar o host 8.8.8.8, conhecido por ser o principal servidor de DNS da Google. Repare que o tempo em milissegundos é maior (o pacote certamente “viajou” mais longe) e, desta vez, limitamos o número de pings a quatro, usando o parâmetro `-c` pois, por padrão no Linux, o teste acontece indefinidamente até que você o interrompa com um CTRL+C.

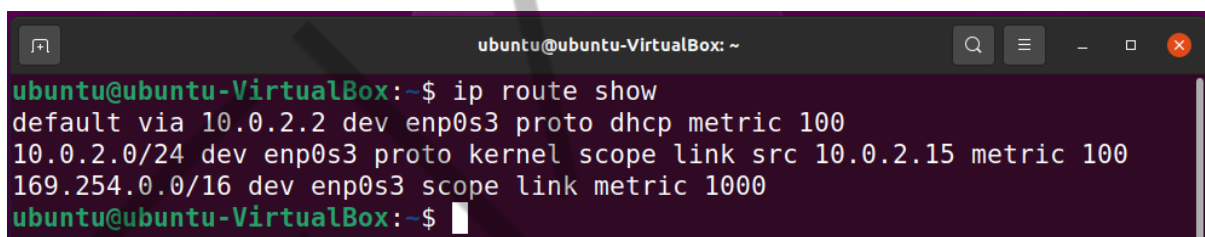
A terminal window titled 'ubuntu@ubuntu-VirtualBox: ~' showing the execution of the command 'ping -c 4 8.8.8.8'. The output shows four successful ping requests to 8.8.8.8 with varying response times (6.99 ms, 6.87 ms, 6.47 ms, 6.66 ms). It also displays ping statistics: 4 packets transmitted, 4 received, 0% packet loss, and an average round-trip time of 6.470 ms.

```
ubuntu@ubuntu-VirtualBox:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=6.99 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=6.87 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=6.47 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=116 time=6.66 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 6.470/6.747/6.991/0.198 ms
ubuntu@ubuntu-VirtualBox:~$
```

Figura 5 – Exemplo de execução do comando ping com parâmetro -c
Fonte: Elaborado pelo autor (2021)

Podemos ver as rotas em ação com o comando `ip route show`, e a primeira linha evidencia que o host 10.0.2.2 da rede do exemplo atual como um gateway padrão (a rota é denominada default), sendo assim, todos os IPv4 que não se enquadrarem nas demais rotas abaixo são automaticamente enviados para esta rota padrão, certamente o caminho em que o echo request de 8.8.8.8 tomou no exemplo anterior.

A terminal window titled 'ubuntu@ubuntu-VirtualBox: ~' showing the output of the command 'ip route show'. The output lists three routes: a default route via 10.0.2.2, a specific route for 10.0.2.0/24, and a specific route for 169.254.0.0/16.

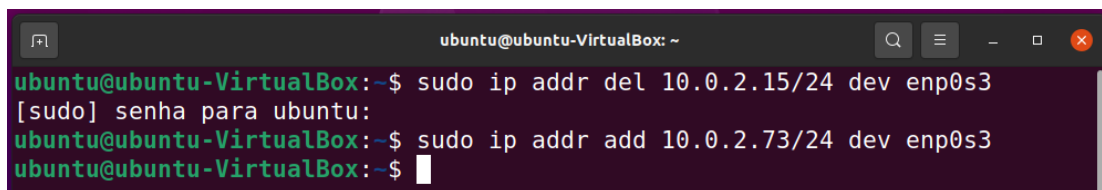
```
ubuntu@ubuntu-VirtualBox:~$ ip route show
default via 10.0.2.2 dev enp0s3 proto dhcp metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
169.254.0.0/16 dev enp0s3 scope link metric 1000
ubuntu@ubuntu-VirtualBox:~$
```

Figura 6 – Exemplo de execução do comando ip route show
Fonte: Elaborado pelo autor (2021)

Vamos efetuar uma troca no endereçamento IPv4 da interface da rede, trocando o final do endereço que é quinze para setenta e três. Para tal, basta utilizar os parâmetros de `del` (delete, remover) e `add` (adicionar) endereços, com permissão especial para isso, é claro.

```
$ sudo ip addr del 10.0.2.15/24 dev enp0s3
$ sudo ip addr add 10.0.2.73/24 dev enp0s3
```

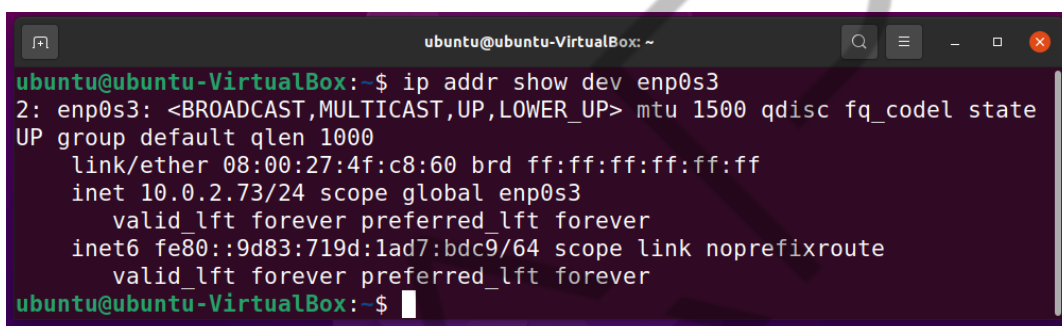
Comando de prompt 2 – Comando para remoção e adição de endereço IPv4 na interface de rede
Fonte: Elaborado pelo autor (2021)

A terminal window titled 'ubuntu@ubuntu-VirtualBox: ~' with search, menu, and window control icons. It shows the execution of two commands: 'sudo ip addr del 10.0.2.15/24 dev enp0s3' followed by a password prompt '[sudo] senha para ubuntu:', and then 'sudo ip addr add 10.0.2.73/24 dev enp0s3'. The prompt returns to the shell after each command.

```
ubuntu@ubuntu-VirtualBox:~$ sudo ip addr del 10.0.2.15/24 dev enp0s3
[sudo] senha para ubuntu:
ubuntu@ubuntu-VirtualBox:~$ sudo ip addr add 10.0.2.73/24 dev enp0s3
ubuntu@ubuntu-VirtualBox:~$
```

Figura 7 – Exemplo de execução de remoção e adição de endereço IPv4
Fonte: Elaborado pelo autor (2021)

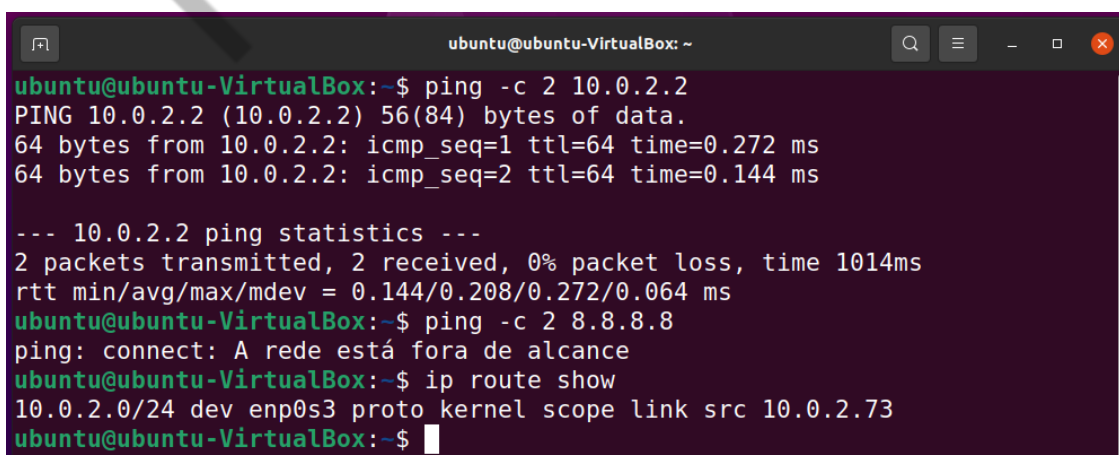
Ao repetir o comando `ip addr show dev enp0s3`, é possível observar que o endereçamento IPv4 mudou mesmo, da forma que desejávamos.

A terminal window titled 'ubuntu@ubuntu-VirtualBox: ~' showing the output of the command 'ip addr show dev enp0s3'. The output displays network details for the 'enp0s3' interface, including MTU, state, group, link/ether address, and the assigned IPv4 address '10.0.2.73/24'.

```
ubuntu@ubuntu-VirtualBox:~$ ip addr show dev enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:4f:c8:60 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.73/24 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::9d83:719d:1ad7:bdc9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-VirtualBox:~$
```

Figura 8 – Exemplo de execução de endereço IPv4 modificado
Fonte: Elaborado pelo autor (2021)

Aproveitamos para repetir os comandos de `ping`, tanto para conectividade interna quando para externa, e é possível observar uma coisa interessante: apenas o host 10.0.2.2 está respondendo, o host 8.8.8.8 passou a ficar fora do alcance. Ao repetir o comando `ip route show`, é possível observar que duas rotas sumiram junto com o comando de remoção de endereço realizado há alguns comandos.

A terminal window titled 'ubuntu@ubuntu-VirtualBox: ~' showing the results of ping tests and the output of 'ip route show'. The ping tests show successful connectivity to 10.0.2.2 and a 'connect: A rede está fora do alcance' (connection refused) for 8.8.8.8. The 'ip route show' command displays the current routing table.

```
ubuntu@ubuntu-VirtualBox:~$ ping -c 2 10.0.2.2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data:
64 bytes from 10.0.2.2: icmp_seq=1 ttl=64 time=0.272 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=64 time=0.144 ms

--- 10.0.2.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1014ms
rtt min/avg/max/mdev = 0.144/0.208/0.272/0.064 ms
ubuntu@ubuntu-VirtualBox:~$ ping -c 2 8.8.8.8
ping: connect: A rede está fora do alcance
ubuntu@ubuntu-VirtualBox:~$ ip route show
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.73
ubuntu@ubuntu-VirtualBox:~$
```

Figura 9 – Testes de conectividade com o comando ping
Fonte: Elaborado pelo autor (2021)

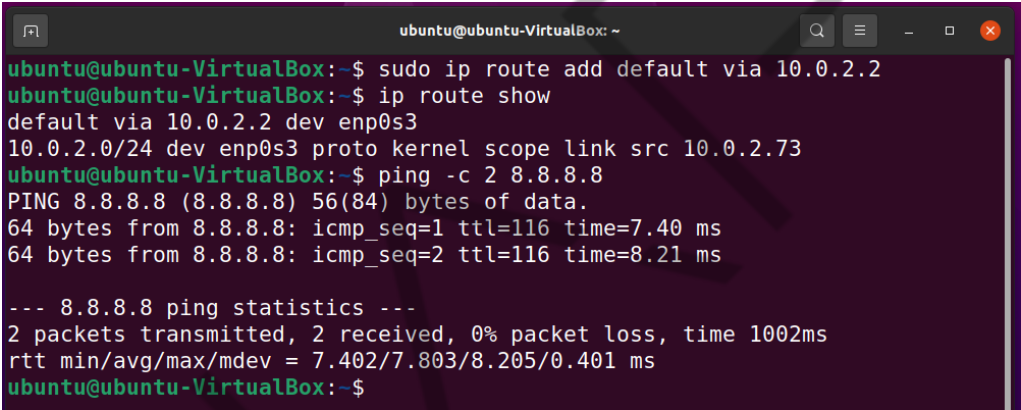
O comando **ip route** a seguir adiciona novamente o *gateway* padrão aos moldes do que foi observado anteriormente, restabelecendo a rota que ali existia.

```
$ sudo ip route add default via 10.0.2.2
```

Comando de prompt 3 – Comando para adição de gateway padrão

Fonte: Elaborado pelo autor (2021)

Ao rodar o comando e testar sua existência e repetir o comando de **ping**, é possível observar que a conectividade ao host externo voltou a acontecer normalmente.



```
ubuntu@ubuntu-VirtualBox:~$ sudo ip route add default via 10.0.2.2
ubuntu@ubuntu-VirtualBox:~$ ip route show
default via 10.0.2.2 dev enp0s3 scope link src 10.0.2.73
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.73
ubuntu@ubuntu-VirtualBox:~$ ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=7.40 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=8.21 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 7.402/7.803/8.205/0.401 ms
ubuntu@ubuntu-VirtualBox:~$
```

Figura 10 – Exemplo de execução e teste de adição de gateway padrão

Fonte: Elaborado pelo autor (2021)

2.2 nmap – Escaneamento de portas

Um dos grandes riscos para um computador conectado a uma rede são as portas de serviço que se encontram abertas, esperando para atender aos serviços de rede. Muitas destas portas são abertas por serviços que, por muitas vezes, não sabemos o porquê de estarem ativos e que portas abrem para tais. O comando **nmap** realiza um escaneamento de portas (o chamado Port Scan) na máquina-alvo, indicando quais portas estão abertas e como estão abertas.

Recomendo que você realize um escaneamento de portas apenas em máquinas das quais você toma conta, já que o ato é considerado um ataque de invasão preliminar. Antes de realizar o escaneamento, certifique-se de que o comando está instalado; caso não esteja, digite `$ sudo apt install nmap`. Para realizar a análise, digite `nmap` e o IP da máquina em questão:

```
hpoyatos@debian:~$ sudo nmap 127.0.0.1
Starting Nmap 7.40 ( https://nmap.org ) at 2018-02-06 23:11 -02
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000040s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
Nmap done: 1 IP address (1 host up) scanned in 1.62 seconds
hpoyatos@debian:~$
```

Figura 11 – Exemplo de execução do comando nmap
Fonte: Elaborado pelo autor (2018)

Repare na figura “Exemplo de execução do comando nmap” que decidi escanear meu próprio equipamento apontando para o loopback (127.0.0.1); o escaneamento-padrão detectou apenas uma porta aberta – a 22 –, geralmente destinada para o serviço de ssh.

No entanto, estou presumindo que o serviço em questão é um servidor de ssh, porque esta é a porta destinada a ele; a verdade é que pode ser qualquer outro serviço “escutando” nesta porta. Para aprimorar esta análise, posso usar o parâmetro `-allports`, que, como a tradução indica, vai escanear todas as portas (o escaneamento padrão despreza algumas). Os parâmetros `-s` e `-V` informam o serviço que está “escutando” e sua versão, respectivamente. Veja o exemplo:

```
hpoyatos@debian:~$ sudo nmap 127.0.0.1 --allports -sV
[sudo] senha para hpoyatos:
Starting Nmap 7.40 ( https://nmap.org ) at 2018-02-06 23:38 -02
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000060s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4p1 Debian 10+deb9u2 (protocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.00 seconds
hpoyatos@debian:~$
```

Figura 12 – Exemplo de execução do comando nmap com análise aprofundada
Fonte: Elaborado pelo autor (2018)

A análise do nmap mostra que, na porta 22, temos um servidor de OpenSSH e a versão exata (7.4p1), também é possível constatar que a versão do sistema operacional (Debian 10+deb9u2) e a versão do protocolo SSH que está rodando (2.0).

É muita informação pertinente; para se proteger, talvez possa desligar um serviço de rede que não esteja sendo utilizado. Para um atacante, a versão exata do OpenSSH se desatualizada, proporciona o uso de um exploit e uma falha de segurança, permitindo uma invasão e até mesmo o controle do sistema com o usuário root. Proteja-se de riscos desnecessários.

3 GERENCIAMENTO DE PROCESSOS

Um programa de computador é algo inerte, “adormecido” no disco rígido de um equipamento. Quando chamamos um comando no terminal ou clicamos em um lançador de programa em uma interface gráfica, ele se torna um processo. Sendo assim, um processo é um programa em execução.

Entretanto, em certas ocasiões os processos podem se comportar de uma maneira inesperada, travando ou consumindo recursos demais (por exemplo, processamento, memória etc.) de uma máquina. Os comandos apresentados a seguir permitirão ao administrador monitorar os processos em andamento, verificando o consumo de recursos e permitindo, até mesmo, “congelar” ou eliminar processos em prol da boa saúde do sistema.

3.1 PS – listar processos

O comando “ps” permite listar os processos atuais de um sistema e é, geralmente, acompanhado pelos parâmetros: “-a”, que mostra os processos de todos os usuários (permissão de root para tal), “-u”, que exibe uma lista detalhada contendo o usuário dono do processo e “-x”, que exibe processos que não estejam associados ao terminal de comandos (cada processo tem uma origem, ao usar o parâmetro, podemos ver processos da interface gráfica, por exemplo, que é considerado pelo sistema um outro terminal). A Figura “Exemplo de execução do comando ps” mostra a execução do comando `sudo ps -au`, visto que a adição de “-x” tornaria a lista longa demais:

```

hpoyatos@debian:~$ sudo ps -au
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
Debian-+  528  0.0  0.1 201400 5728 tty1    Ssl+ 07:20   0:00 /usr/lib/gdm3/gdm-x-session gnom
root      534  0.0  1.0 339072 41260 tty1    Sl+   07:20   0:00 /usr/lib/xorg/Xorg vt1 -displayf
Debian-+  562  0.0  0.3 620764 12528 tty1    Sl+   07:20   0:00 /usr/lib/gnome-session/gnome-ses
Debian-+  584  0.0  4.0 1777036 162292 tty1    Sl+   07:20   0:04 /usr/bin/gnome-shell
Debian-+  612  0.0  0.7 1159620 28640 tty1    Sl+   07:21   0:00 /usr/lib/gnome-settings-daemon/g
hpoyatos  722  0.0  0.1 201400 5792 tty2    Ssl+ 07:21   0:00 /usr/lib/gdm3/gdm-x-session --ru
root      724  0.2  1.3 363428 53920 tty2    Sl+   07:21   0:14 /usr/lib/xorg/Xorg vt2 -displayf
hpoyatos  732  0.0  0.3 620920 12860 tty2    Sl+   07:21   0:00 /usr/lib/gnome-session/gnome-ses
hpoyatos  810  0.6  5.5 1824532 225060 tty2    Rl+   07:21   0:43 /usr/bin/gnome-shell
hpoyatos  909  0.0  0.9 1144416 37256 tty2    Sl+   07:21   0:00 /usr/lib/gnome-settings-daemon/g
hpoyatos  932  0.0  0.2 445036 11920 tty2    SNl+ 07:21   0:00 /usr/lib/tracker/tracker-miner-a
hpoyatos  933  0.0  0.3 595832 15976 tty2    SNl+ 07:21   0:00 /usr/lib/tracker/tracker-extract
hpoyatos  935  0.0  0.4 505968 16528 tty2    SNl+ 07:21   0:00 /usr/lib/tracker/tracker-miner-f
hpoyatos  936  0.0  1.7 920076 71644 tty2    Sl+   07:21   0:01 /usr/bin/gnome-software --gappli
hpoyatos  939  0.0  0.3 418032 13420 tty2    SNl+ 07:21   0:00 /usr/lib/tracker/tracker-miner-u
hpoyatos  940  0.0  1.2 1015060 51412 tty2    Sl+   07:21   0:00 /usr/lib/evolution/evolution-ala
hpoyatos  949  0.0  0.3 495392 12904 tty2    Sl+   07:21   0:00 /usr/lib/gnome-settings-daemon/g
hpoyatos 1172  0.0  0.1 20992 4736 pts/0    Ss   07:21   0:00 bash
root      1725  0.0  0.0 55552 3820 pts/0    S+   09:19   0:00 sudo ps -au
root      1726  0.0  0.0 38304 3376 pts/0    R+   09:19   0:00 ps -au
hpoyatos@debian:~$

```

Figura 13 – Exemplo de execução do comando ps
 Fonte: Elaborado pelo autor (2018)

Graças ao parâmetro “-u”, o comando retorna várias informações pertinentes, destacando-se o dono do processo (1ª coluna), o PID (2ª coluna) – um número que identifica unicamente o processo e lhe é concedido no momento de sua criação –, percentual de processamento (CPU) utilizado pelo processo naquele momento (3ª coluna), o percentual ocupado na memória (4ª coluna), a qual terminal (TTY) o processo está atrelado (7ª coluna) e o tempo que o processo está rodando (10ª coluna).

3.2 Top – monitor de processos

O comando **top** não é apenas um monitor de processos, ele permite obter informações importantes, como o total de processos em andamento, seus estados no momento (executando, em espera, parados e *zombies*), uso de CPU, memória RAM e memória de *swap*.

Um processo pode estar em espera para uso de processos ou I/O (que o Linux chama de sleeping), sendo executado no processador (chamado aqui de running), congelado ou bloqueado por alguma razão (stopped) ou trata-se de um processo zumbi. Zumbi? Como é isso? Bem, trata-se de processos que já foram completados por alguma razão, contudo, não foram devidamente destruídos. Existe uma hierarquia de processos, já que o processador pode iniciar novos processos (chamados de processos filhos). Isso pode ser facilmente observado quando iniciamos um Apache HTTP Server, o mais utilizado serviço de web server da internet. Quando o número de requisições por páginas *web* começa a subir, o próprio processo do Apache “dá cria”, e cada “processo filhote” cuida de um número fixo de requisições; quando o alto número de requisições começa a cair, o processo-pai vai destruindo seus filhos, liberando recursos do servidor. Bem, nem sempre esse verdadeiro “balé sincronizado” dá certo, e quando um filho perde sua função, entretanto, não é devidamente destruído, ele se torna um zumbi.



Figura 14 – In your hands, zombie
Fonte: Shutterstock (2019)

Bem, mas voltando ao comando top. Encabeçando a lista, estão sempre os processos que consomem mais recursos de sistema e, como podemos ver na figura “Exemplo de execução do top”, os processos associados à interface gráfica (gnome-shell, Xorg etc.) são os mais “fominhas” e estão sempre no topo:

```

top - 09:27:51 up 2:07, 1 user, load average: 0,00, 0,01, 0,00
Tasks: 127 total, 1 running, 126 sleeping, 0 stopped, 0 zombie
%Cpu(s): 6,4 us, 0,3 sy, 0,0 ni, 93,3 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
KiB Mem : 4051036 total, 3007780 free, 546880 used, 496376 buff/cache
KiB Swap: 4192252 total, 4192252 free, 0 used. 3281896 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
810	hpoyatos	20	0	1824532	225224	84472	S	4,3	5,6	0:48.08	gnome-shell
724	root	20	0	360996	51484	28964	S	2,3	1,3	0:15.82	Xorg
1166	hpoyatos	20	0	601272	33604	25084	S	0,3	0,8	0:03.24	gnome-terminal-
1756	hpoyatos	20	0	44908	3744	3116	R	0,3	0,1	0:00.23	top
1	root	20	0	139000	6824	5268	S	0,0	0,2	0:00.82	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.05	ksoftirqd/0
5	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	kworker/0:0H
7	root	20	0	0	0	0	S	0,0	0,0	0:00.13	rcu_sched
8	root	20	0	0	0	0	S	0,0	0,0	0:00.00	rcu_bh
9	root	rt	0	0	0	0	S	0,0	0,0	0:00.00	migration/0
10	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	lru-add-drain
11	root	rt	0	0	0	0	S	0,0	0,0	0:00.02	watchdog/0
12	root	20	0	0	0	0	S	0,0	0,0	0:00.00	cpuhp/0
13	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kdevtmpfs
14	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	netns
15	root	20	0	0	0	0	S	0,0	0,0	0:00.00	khungtaskd

Figura 15 – Exemplo de execução do top
 Fonte: Elaborado pelo autor (2018)

Conforme mencionado antes, é por esta razão que vários servidores Linux sequer possuem interface gráfica, economizando processamento e armazenamento valiosos... por isso é tão importante conhecer a linha de comando!

O comando trava o terminal, assim, a lista permanece sendo exibida dinamicamente (os processos vão surgindo, usando mais ou menos recursos e são destruídos) pelo tempo que for necessário. Para sair do comando top, digite q (quit, sair).

3.3 Kill e Killall – matando processos

Eventualmente, um administrador de sistema precisa interromper processos em andamento de maneira abrupta, seja por estarem consumindo recursos indesejados, seja por representarem algum risco ao sistema. E não vamos esquecer dos zumbis!

Para demonstrar seu funcionamento, a interface gráfica pode ser aproveitada e o aplicativo GNOME Chess, que faz parte do pacote padrão para desktop do Debian. Ao executar \$ ps aux (o traço é opcional nesse caso), o processo é o penúltimo a ser exibido, já que o ps é o último (vide figura “Exemplo de execução do comando ps aux”):


```

hpoyatos@debian: ~
Arquivo Editar Ver Pesquisar Terminal Ajuda
hpoyatos 1125 0.0 0.9 1144416 37668 tty2 Sl+ 16:31 0:01 /usr/lib/gnome-
hpoyatos 1142 0.0 0.7 852216 32248 ? Ssl 16:31 0:00 /usr/lib/evolut
hpoyatos 1147 0.0 0.3 445032 12252 tty2 SNL+ 16:31 0:00 /usr/lib/tracke
hpoyatos 1149 0.0 0.4 597928 16256 tty2 SNL+ 16:31 0:00 /usr/lib/tracke
hpoyatos 1151 0.0 0.3 579624 16184 tty2 SNL+ 16:31 0:00 /usr/lib/tracke
hpoyatos 1152 0.0 2.3 944012 93568 tty2 Sl+ 16:31 0:01 /usr/bin/gnome-
hpoyatos 1153 0.0 0.3 418032 13616 tty2 SNL+ 16:31 0:00 /usr/lib/tracke
hpoyatos 1154 0.0 1.3 1088992 52944 tty2 Sl+ 16:31 0:00 /usr/lib/evolut
hpoyatos 1160 0.0 0.4 396028 18836 ? Ssl 16:31 0:00 /usr/lib/tracke
hpoyatos 1172 0.0 0.3 495392 12624 tty2 Sl+ 16:31 0:00 /usr/lib/gnome-
hpoyatos 1184 0.0 0.6 809088 24640 ? Sl 16:31 0:00 /usr/lib/evolut
hpoyatos 1197 0.0 0.1 187292 4716 ? Sl 16:31 0:00 /usr/lib/dconf/
hpoyatos 1198 0.0 0.5 777356 20880 ? Sl 16:31 0:00 /usr/lib/evolut
hpoyatos 1200 0.0 0.5 709256 22580 ? Ssl 16:31 0:00 /usr/lib/evolut
hpoyatos 1216 0.0 0.5 799392 21956 ? Sl 16:31 0:00 /usr/lib/evolut
hpoyatos 1295 0.0 0.1 193752 5464 ? Ssl 16:31 0:00 /usr/lib/gvfs/g
root 2718 0.0 0.1 20472 4408 ? S 18:01 0:00 /sbin/dhclient
root 2752 0.0 0.0 0 0 ? S 18:50 0:00 [kworker/0:1]
root 2754 0.0 0.0 0 0 ? S 18:55 0:00 [kworker/0:0]
hpoyatos 2778 0.2 0.8 600956 32964 ? Ssl 18:56 0:00 /usr/lib/gnome-
hpoyatos 2784 0.0 0.1 20992 4900 pts/0 Ss 18:56 0:00 bash
hpoyatos 2829 0.3 0.8 482820 33724 tty2 Sl+ 18:57 0:00 gnome-chess
hpoyatos 2839 0.0 0.0 38304 3164 pts/0 R+ 18:59 0:00 ps aux
hpoyatos@debian:~$

```

Figura 16 – Exemplo de execução do comando ps aux
Fonte: Elaborado pelo autor (2018)

Trata-se de um bom momento para demonstrar a possibilidade de combinar comandos: a execução do comando `ps`, especialmente com os parâmetros “aux”, pode ficar longa e confusa. O símbolo “|” (ou pipe) permite combinar um comando a outro. Observe o que acontece quando combino a saída do comando `ps aux` (a lista em si) com o comando `grep` (que recebe a lista como entrada), que permite busca por palavras-chave:

```

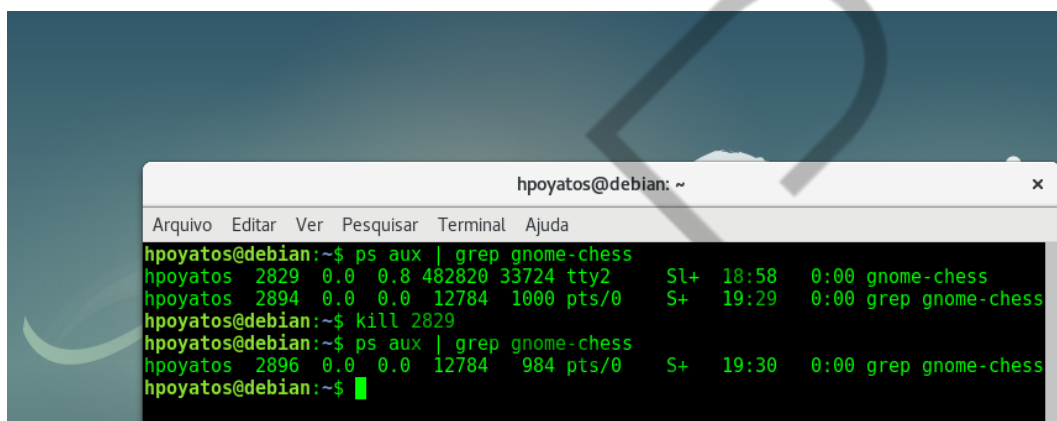
hpoyatos@debian: ~
Arquivo Editar Ver Pesquisar Terminal Ajuda
hpoyatos@debian:~$ ps aux | grep gnome-chess
hpoyatos 2829 0.0 0.8 482820 33724 tty2 Sl+ 18:58 0:00 gnome-chess
hpoyatos 2894 0.0 0.0 12784 1000 pts/0 S+ 19:29 0:00 grep gnome-chess
hpoyatos@debian:~$

```

Figura 17 – Exemplo de execução do comando ps aux | grep gnome-chess
Fonte: Elaborado pelo autor (2018)

A lista de processos mostra apenas duas linhas: o processo `gnome-chess` e a própria execução do comando `grep` (que, afinal, possui a palavra-chave). Muito interessante, não é?

Para “matar” o processo GNOME Chess, usamos o sugestivo comando `kill` (que, em inglês, significa justamente “matar”), acompanhado do identificador do processo (PID), informado na segunda coluna do comando `ps aux` (no exemplo da figura “Exemplo de execução do comando `ps aux | grep gnome-chess`”, o PID é 2829):



```
hpyatos@debian: ~  
Arquivo Editar Ver Pesquisar Terminal Ajuda  
hpyatos@debian:~$ ps aux | grep gnome-chess  
hpyatos 2829 0.0 0.8 482820 33724 tty2 Sl+ 18:58 0:00 gnome-chess  
hpyatos 2894 0.0 0.0 12784 1000 pts/0 S+ 19:29 0:00 grep gnome-chess  
hpyatos@debian:~$ kill 2829  
hpyatos@debian:~$ ps aux | grep gnome-chess  
hpyatos 2896 0.0 0.0 12784 984 pts/0 S+ 19:30 0:00 grep gnome-chess  
hpyatos@debian:~$
```

Figura 18 – Exemplo de execução do comando `kill`
Fonte: Elaborado pelo autor (2018)

Além do fato do tabuleiro de xadrez ser fechado abruptamente, ao repetir o comando de listagem (`ps`) à procura de `gnome-chess`, o processo não existe mais; repare que não foi necessário o uso do `sudo`, uma vez que o processo tinha meu próprio usuário como dono (primeira coluna do comando `ps aux`).

Embora o comando se chame `kill` e possua esse potencial sanguinário, ele pode ser utilizado para mandar sinais diferentes ao processo. Por exemplo, o uso do parâmetro `-SIGSTOP` no comando `kill` não destruiria o processo, apenas o pausaria ou congelaria (e se torna um dos processos stopped lá do top, lembra?). Sendo assim, um processo com a compressão de um arquivo (comando `tar`) que demorasse muito tempo não precisaria necessariamente ser interrompido, caso fosse necessário liberar recursos de sistema, bastaria dar uma pausada nele. O parâmetro `-SIGCONT` descongela o processo, que retoma seu processamento exatamente de onde parou.

```
$ kill -SIGSTOP 2829
```

Comando de prompt 4 – Exemplo do comando `kill` congelando o processo de PID 2829
Fonte: Elaborado pelo autor (2018)

Esclarecendo, a omissão do parâmetro manda um sinal de SIGKILL que, na prática, significa a destruição do processo.

Há uma variação do comando kill, conhecida como killall (ou “matar todos”), que deve ser acompanhada pelo nome do processo (e não o PID). Isso acontece pois, como o nome sugere, o comando pode destruir todos os processos que possuam o mesmo nome.



Figura 19 – Comando killall em ação

Fonte: <https://www.slashfilm.com/rambo-5-development/> (2018)

Para a demonstração, usei o servidor web mais utilizado do mundo, o Apache HTTP Server; caso precise de um servidor web, não deixe de instalá-lo com o comando `$ sudo apt install apache2`. O apache é útil neste exemplo, uma vez que, conforme foi explicado há pouco, trata-se de um serviço escalonável, ele cria subprocessos conforme a necessidade de atender mais requisições de rede aumenta. Seu processo principal gera outros dois processos, assim, temos três processos chamados apache2 (vide figura “Exemplo de execução do comando killall”):

```
hpoyatos@debian:~$ sudo /etc/init.d/apache2 start
[ ok ] Starting apache2 (via systemctl): apache2.service.
hpoyatos@debian:~$ ps aux | grep apache2
root      3793  0.0  0.1 75608 4368 ?        Ss   19:44   0:00 /usr/sbin/apache
2 -k start
www-data  3796  0.0  0.1 364768 4060 ?        Sl   19:44   0:00 /usr/sbin/apache
2 -k start
www-data  3797  0.0  0.1 364768 4060 ?        Sl   19:44   0:00 /usr/sbin/apache
2 -k start
hpoyatos  3868  0.0  0.0 12784   952 pts/0    S+   19:44   0:00 grep apache2
hpoyatos@debian:~$ sudo killall apache2
hpoyatos@debian:~$ ps aux | grep apache2
hpoyatos  3880  0.0  0.0 12784   948 pts/0    S+   19:44   0:00 grep apache2
hpoyatos@debian:~$
```

Figura 20 – Exemplo de execução do comando killall

Fonte: Elaborado pelo autor (2018)

O primeiro comando do exemplo acima (`$ sudo /etc/init.d/apache2 start`) é a inicialização do daemon do apache2, o gerenciador de serviço; ou seja, é assim que um serviço no Linux é inicializado. Ao executar `$ ps aux | grep apache2`, repare que possui três processos, sendo um principal e dois subprocessos (embora não saiba dizer qual é qual). O comando `killall apache2` é acompanhado pelo `sudo`, visto que o serviço pertence ao usuário `www-data` e, como podemos ver, ao listar os processos novamente, o comando faz o serviço sem deixar vestígios.



Figura 21 – Melhor exemplo da sutileza (ao não deixar vestígios) do `killall`
Fonte: Shutterstock (2019)

Outra forma de interromper processos é fechar o terminal no qual eles rodam. Processos não possuem apenas um usuário dono, mas um terminal (seja ele gráfico ou não) associado a eles.

3.4 & – Rodando processos em background

Todo comando que executa em um terminal, ao seu término devolve o prompt de comando para a execução de novos comandos, todavia, até o seu encerramento, ele “trava” o terminal, não permitindo novas interações do usuário. Embora seja um fenômeno comum em comandos de modo texto, esse comportamento torna-se mais evidente quando utiliza-se o terminal para chamar aplicativos gráficos, como o `gnome-chess`:

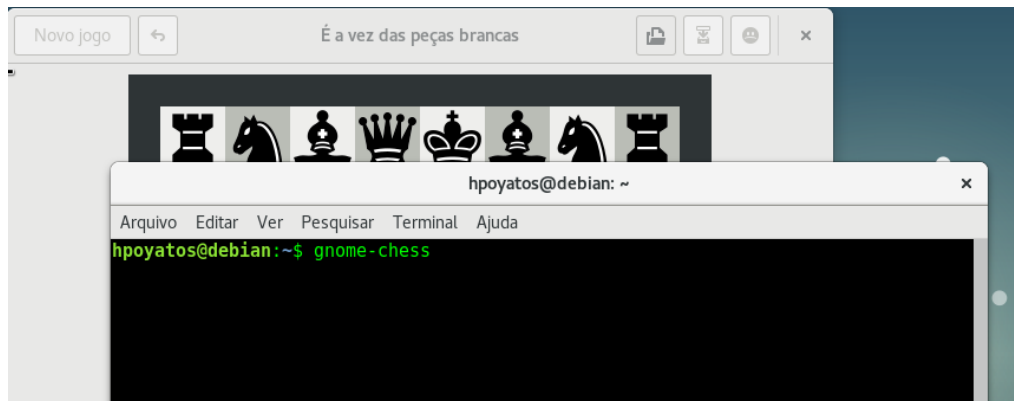


Figura 22 – Exemplo de execução do comando GNOME Chess travando o terminal de comandos
Fonte: Elaborado pelo autor (2018)

Isso acontece porque o processo do GNOME Chess está rodando em foreground ou primeiro plano. Podemos solicitar que o processo execute em segundo plano ou background, desde que a linha de comando que crie o processo seja acompanhada por um “&” (e comercial) em seu final:

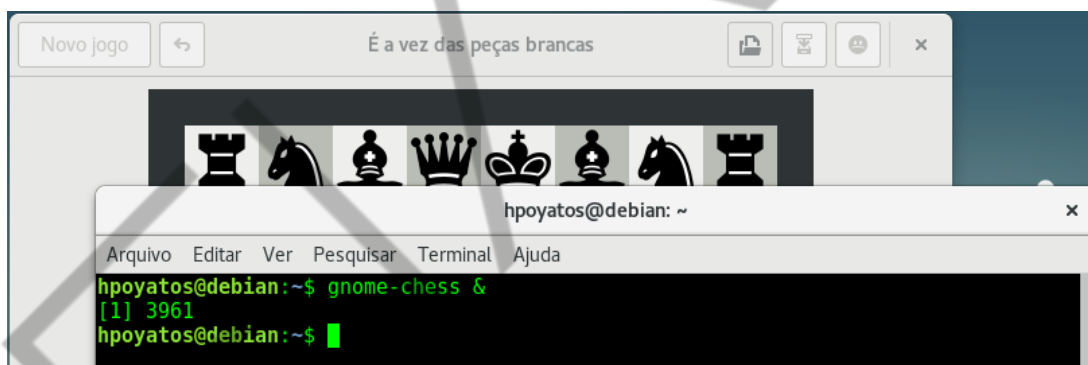


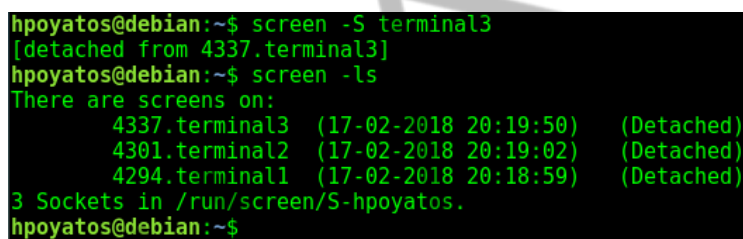
Figura 23 – Exemplo de execução do comando GNOME Chess rodando em *background*
Fonte: Elaborado pelo autor (2018)

Observe a figura “Exemplo de execução do comando GNOMR Chess rodando em background” e repare que o prompt de comando é devolvido, informando antes o identificador do processo (PID) que, neste caso, é 3961. No entanto, o fato de o processo rodar em segundo plano não significa que este está livre do terminal; o processo continua atrelado a ele e, se for fechado, leva o tabuleiro de xadrez com ele.

3.5 Screen – criando terminais virtuais

Processos sempre estarão associados a terminais, isto é um fato. Porém, há formas de criar terminais virtuais que podem ser abandonados e retomados em um segundo momento. Isso é particularmente interessante, nos casos em que precisamos rodar comandos muito longos, como a realização de um backup (como tar ou outro comando) ou o download de um arquivo muito grande (com o bom e velho wget). Imagine, ainda, que você queira iniciar o comando fisicamente no servidor da empresa e deseja verificar a execução horas depois, remotamente, de sua casa? O comando **screen** pode ajudar muito nesses casos.

Não deixe de instalá-lo com `$ sudo apt install screen`. O comando `screen -S [nome do terminal]` cria um terminal virtual que pode ser abandonado com o atalho de teclado [CTRL+A], seguido da tecla [D]:



```
hpoyatos@debian:~$ screen -S terminal3
[detached from 4337.terminal3]
hpoyatos@debian:~$ screen -ls
There are screens on:
  4337.terminal3  (17-02-2018 20:19:50)  (Detached)
  4301.terminal2  (17-02-2018 20:19:02)  (Detached)
  4294.terminal1  (17-02-2018 20:18:59)  (Detached)
3 Sockets in /run/screen/S-hpoyatos.
hpoyatos@debian:~$
```

Figura 24 – Exemplo de execução do comando `screen -ls`
Fonte: Elaborado pelo autor (2018)

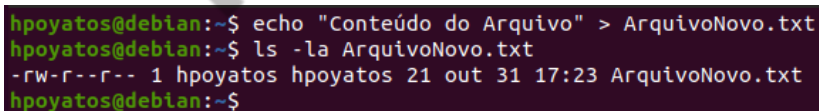
A Figura “Exemplo de execução do comando `screen -ls`” mostra um terceiro terminal que foi criado e abandonado com [CTRL+A] e [D]. Ao executar `screen -ls`, podemos observar três terminais disponíveis (os dois primeiros foram criados antes, com o mesmo comando `screen -S <nome>`). O comando `$ screen -r terminal2` que permite retornar ao terminal `4301.terminal2`, e digitar “exit” nesse terminal, encerrando sua execução.

4 PERMISSÕES: PODE OU NÃO PODE?

Até o momento, falamos bastante sobre a questão do usuário no Linux possuir ou não permissão de fazer alguma coisa e, quando ele não a tinha, recorriamos ao `sudo` para que ele tivesse a permissão necessária. Contudo, é natural em sistemas de arquivos a necessidade de mudar as permissões de arquivos ou pastas, de forma que usuários comuns (que não possam apelar para o `sudo`) tenham as permissões necessárias para executar suas tarefas no sistema.

Abordaremos, na sequência, os comandos que permitem mudar o dono do arquivo (`chown`), o grupo dono de arquivo (`chgrp`) e as permissões propriamente ditas (`chmod`). Optaremos por abordar, aqui, o mecanismo de permissões herdado do Unix, mas saiba que existem possibilidades mais complexas de gerenciar permissionamento (se quiser saber mais sobre isso, pesquise por FUSE, *Filesystem in Userspace*).

Antes de tudo, vamos criar um arquivo que sirva à demonstração. Na Figura “Criação simples de um novo arquivo e sua listagem” elaboramos um arquivo da maneira mais simples possível, sem usar um editor de textos: o comando `echo` ecoa ou propaga a mensagem que coloquei entre aspas e o símbolo “>” direciona a saída padrão do comando (que aconteceria no terminal) para dentro de um arquivo chamado `ArquivoNovo.txt`. Como o arquivo não existia, o comando prontamente o criou. É importante ressaltar que isso só foi possível pois foi feito dentro do meu diretório de usuário, no qual possuo permissões de criação. O comando `ls -la ArquivoNovo.txt` permite o detalhamento do arquivo recém-criado.



```
hpoyatos@debian:~$ echo "Conteúdo do Arquivo" > ArquivoNovo.txt
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-r--r-- 1 hpoyatos hpoyatos 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$
```

Figura 25 – Criação simples de um novo arquivo e sua listagem
Fonte: Elaborado pelo autor (2019)

Repare nas primeiras colunas do comando “ls -la”: começando pela terceira e quartas colunas, temos o termo “hpoyatos”. Na terceira coluna, observamos o nome do usuário dono desse arquivo: como acabei de criá-lo, nada mais justo que eu seja considerado seu dono. A quarta coluna, também “hpoyatos”, corresponde ao grupo dono do arquivo. Quando criamos um usuário no Linux com `useradd`, o comando também cria um grupo de mesmo nome que o usuário, e o coloca como membro único. Sendo assim, geralmente em um sistema Linux temos usuários e grupos homônimos.

Vamos agora para a primeira coluna: ela sempre possui 10 caracteres, sendo o primeiro para informar o tipo do arquivo (se é um **d**iretório, um **l**ink ou atalho, dispositivo que funciona com **c**aracteres ou **b**inários, ou é um simples arquivo mesmo, mostrado com um tracinho “-”) e os nove seguintes indicam as permissões.

O primeiro trio informa as permissões do dono do arquivo: a presença da letra **r** indica o direito à sua leitura (*read*), a letra **w** mostra a possibilidade de escrita (*write*) e o “**x**”, de *execute*, refere-se à sua execução (se for um arquivo) e acesso, se for um diretório. Repare que, no primeiro trio do exemplo, temos “**rw-**”, o que indica que posso ler e escrever no arquivo, mas não posso executá-lo (ausência de **x**).

O segundo trio possui a mesma lógica, contudo, se aplica ao grupo dono do arquivo. No exemplo em questão, temos “**r--**”, indicando que os membros do grupo hpoyatos só podem ler o arquivo, mas não podem modificá-lo ou executá-lo (ausência de **w** e **x**). Neste caso em específico, a permissão de escrita como dono do usuário prevalece à sua ausência na permissão de grupo, ou seja, os direitos de escrita por fazer do grupo não são perdidos.

O terceiro trio se refere às permissões de todos os demais que não são donos ou não fazem parte do grupo dono do arquivo. No exemplo do ArquivoNovo.txt, todo mundo pode ler o arquivo (“**r--**”).

Agora que já repassamos as informações de permissão presentes na listagem de arquivos, vamos aos comandos!

4.1 chown

O comando **chown** (abreviação de *change owner*) permite mudar o dono do arquivo e, para tal, basta digitar `chown` seguido do nome do novo dono e o arquivo ao qual o comando se aplica. Conforme pode ser visto na Figura “Mudando o dono do arquivo com chown”, o comando exige privilégio especial e deve ser usado com `sudo`. O exemplo mostra a listagem antes e depois, evidenciando a mudança do dono.

```
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-r--r-- 1 hpoyatos hpoyatos 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$ chown fiap ArquivoNovo.txt
chown: alterando o dono de 'ArquivoNovo.txt': Operação não permitida
hpoyatos@debian:~$ sudo chown fiap ArquivoNovo.txt
[sudo] senha para hpoyatos:
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-r--r-- 1 fiap hpoyatos 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$
```

Figura 26 – Mudando o dono do arquivo com chown
Fonte: Elaborado pelo autor (2019)

O comando **chown** permite ainda alterar o dono do arquivo e o grupo dono do arquivo em uma só tacada. Para isso, basta digitar `chown <dono>:<grupo> <arquivo>`, um exemplo desse uso pode ser observado na Figura “Mudando o dono do arquivo e grupo dono com chown”.

```
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-r--r-- 1 fiap hpoyatos 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$ sudo chown hpoyatos:fiapon ArquivoNovo.txt
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-r--r-- 1 hpoyatos fiapon 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$
```

Figura 27 – Mudando o dono do arquivo e grupo dono com chown
Fonte: Elaborado pelo autor (2019)

O curinga “*” pode ser utilizado para se referir a todos os arquivos e diretórios, quando precisar mudar a posse de vários arquivos ao mesmo tempo. O parâmetro “-r” indica recursividade, ou seja, usar o parâmetro indica alterar o dono de todos os arquivos presentes em todos os subdiretórios. Utilize o comando com sabedoria.

```
$ sudo chown fiap * -r
```

Comando de prompt 5 – Dando ao usuário “fiap” a posse de todos os arquivos e subdiretórios
Fonte: Elaborado pelo autor (2019)

4.2 chgrp

O comando **chgrp** (abreviação de *change group*) é utilizado para trocar o grupo dono do arquivo, como pode ser observado na figura “Mudando o dono do arquivo e grupo dono com chgrp”. O uso de curingas no destino do comando (como o asterisco), assim como o parâmetro “-r”, também está disponível.

```
hpoyatos@debian:~$ sudo groupadd galera
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-r--r-- 1 hpoyatos fiapon 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$ sudo chgrp galera ArquivoNovo.txt
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-r--r-- 1 hpoyatos galera 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$
```

Figura 28 – Mudando o grupo dono do arquivo com chgrp
Fonte: Elaborado pelo autor (2019)

4.3 chmod

Por fim, vejamos o comando **chmod** (abreviação de *change mode*), que permite modificar as permissões de um arquivo ou diretório. Sua sintaxe é digitar o comando `sudo chmod <permissões> <destino>`.

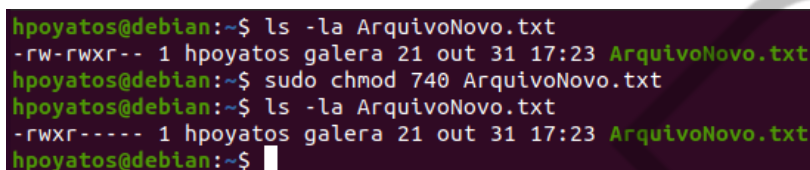
O jeito mais simples de informar as permissões é utilizar as letras o (para *owner* ou dono), g (para *group* ou grupo) e a (para *all* ou todos), seguidas por um símbolo de “+”, para conceder permissões, ou “-”, para revogar, e a permissão propriamente dita “r” (para *read* ou leitura), “w” (para *write* ou escrita) e “x” (para *execute* ou execução). A Figura “Mudando as permissões de grupo com o comando chmod” mostra um exemplo de uso.

```
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-r--r-- 1 hpoyatos galera 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$ sudo chmod g+wx ArquivoNovo.txt
[sudo] senha para hpoyatos:
hpoyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-rwxr-- 1 hpoyatos galera 21 out 31 17:23 ArquivoNovo.txt
hpoyatos@debian:~$
```

Figura 29 – Mudando as permissões de grupo com o comando chmod
Fonte: Elaborado pelo autor (2019)

No exemplo em questão, foram adicionadas as permissões de escrita e execução para o grupo dono do arquivo (no caso, “galera”). A partir de agora, o grupo possui todas as permissões.

Outra maneira de modificar permissões é passá-las para o formato numérico. Nesse caso, a permissão de leitura vale 4, a de escrita 2 e a de execução, 1, e devemos passar as permissões de dono, grupo e todos de uma vez só. A Figura “Mudando as permissões gerais com o comando chmod” mostra um exemplo de uso.



```
hpooyatos@debian:~$ ls -la ArquivoNovo.txt
-rw-rwxr-- 1 hpooyatos galera 21 out 31 17:23 ArquivoNovo.txt
hpooyatos@debian:~$ sudo chmod 740 ArquivoNovo.txt
hpooyatos@debian:~$ ls -la ArquivoNovo.txt
-rwxr----- 1 hpooyatos galera 21 out 31 17:23 ArquivoNovo.txt
hpooyatos@debian:~$
```

Figura 30 – Mudando as permissões gerais com o comando chmod

Fonte: Elaborado pelo autor (2019)

Nesse exemplo, o dono passa a ter todas as permissões ($r+w+x$, $4+2+1=7$), o grupo dono fica apenas com leitura ($r=4$) e os demais não poderão fazer absolutamente nada (0, ausência de permissões).

5 ACESSO A MÁQUINAS REMOTAS COM SSH

Não há necessidade mais atual do que o acesso remoto a uma máquina. A obrigação de estar fisicamente presente em um terminal, em frente ao equipamento que é necessário manipular, é absolutamente incompatível com os dias de hoje, uma realidade totalmente *Cloud Computing*, ou seja, por vezes, sequer sabemos em qual gabinete (ou quais) localiza-se o sistema que estamos utilizando. Dessa maneira, saberá exatamente do que estou falando.

Embora pareça algo muito recente, há décadas temos servidores sem monitores, teclados e mouses para chamar de seus, sendo necessário acessá-los de outro equipamento (bem, se voltar alguns anos mais, nem mouse existia). Terminais sem processamento, apenas com o monitor e teclado, eram destinados para tal fim e eram chamados de “terminais burros”.

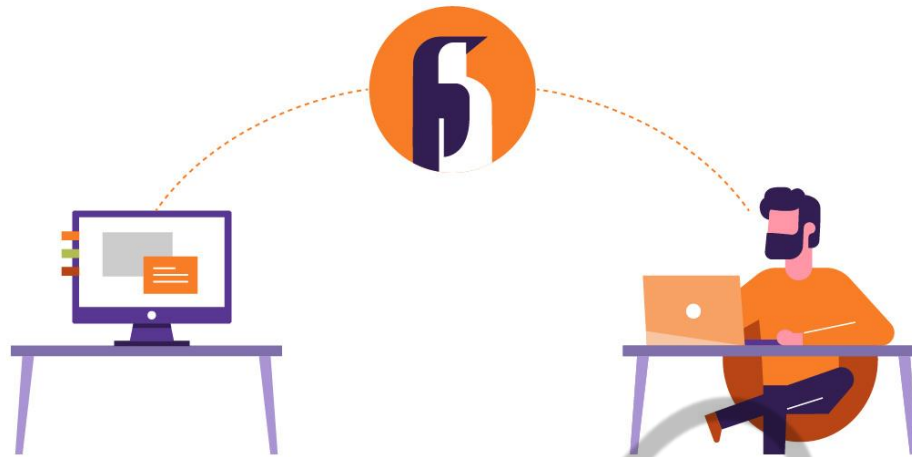


Figura 31 – O pinguim remoto
Fonte: Lynda.com (2016), adaptado por FIAP (2018)

Tão lendário quanto esse paradigma é o serviço que possibilitava isso no Unix, o **telnet**. Ele leva esse nome por possibilitar, ainda, um terminal via linha telefônica discada (sim, você pode ligar para seu servidor). Infelizmente, o serviço se tornou obsoleto pela falta de segurança: os comandos trafegam abertamente em uma rede, sem criptografia, o que tornava a comunicação suscetível a um ataque conhecido como *session hijacking*, no qual o atacante sequestra a sessão como servidor, ao derrubar o cliente e se passar por ele (afinal, ele ficou um bom tempo “escutando” e sabe exatamente como o cliente se comporta).

O SSH (Secure Shell ou shell seguro) é a evolução do telnet, permitindo toda a praticidade de um acesso remoto, com uma diferença: um túnel criptografado usando a criptografia assimétrica ou de chave público-privada garantindo a privacidade e evitando ataques.

5.1 SSH – acesso a outra máquina remotamente

O comando `SSH` é o cliente para acesso a terminais remotos e seu uso é muito simples: ele deve ser seguido do usuário que deseja usar para conectar, “@” (arroba) o nome ou IP do host. Caso o serviço de SSH não esteja em sua porta padrão (a 22), o parâmetro “-p” pode ser adicionado para indicar o número da porta.

Na demonstração a seguir, utilizei um Raspberry Pi como host remoto. O equipamento está munido com um Raspian OS, que é um derivado do Debian específico para esse tipo de hardware. O sistema remoto possui um serviço de SSH devidamente configurado com o OpenSSH, o mais utilizado do mercado, e está disponível na sua porta padrão do serviço – a porta 22. O endereço IP fixo do equipamento é local, em 192.168.0.100. Contudo, é importante observar que isso foi feito apenas a título de curiosidade, pois o serviço de SSH está disponível para qualquer sistema operacional e arquitetura que queira suportar o serviço.



Figura 32 – Um Raspberry Pi 3 model B, um pequeno notável
Fonte: Raspberrypi.org (2018)

Ao acessar pela primeira vez o *host* remoto, este apresenta sua *fingerprint*, que, na prática, é a apresentação de sua chave pública. O serviço de SSH utiliza uma criptografia assimétrica, também conhecida como chave público-privada, a partir da qual cada host gera um par de chaves, distribuindo a chave pública e guardando a privada para si. As informações do terminal trafegam criptografadas, utilizando a chave privada do host de origem e a pública do remoto, e só podem ser descriptografadas pela combinação inversa (a chave pública do host de origem e a privada do remoto) tornando esse tipo de acesso prático e seguro.

```
hpoyatos@debian:~$ ssh pi@192.168.0.100
The authenticity of host '192.168.0.100 (192.168.0.100)' can't be established.
ECDSA key fingerprint is SHA256:KDxCNmmIVYp7edSLHgoEDE6d61ZuuWg74jfn45+WByw.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.0.100' (ECDSA) to the list of known hosts.
pi@192.168.0.100's password:
Linux raspberrypi 4.9.59-v7+ #1047 SMP Sun Oct 29 12:19:23 GMT 2017 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Feb 17 20:29:39 2018 from 192.168.0.10
pi@raspberrypi:~ $
```

Figura 33 – Exemplo de execução do comando SSH
Fonte: Elaborado pelo autor (2018)

A Figura “Exemplo de execução do comando SSH” mostra que a chave pública do Raspberry é aceita e, na sequência, é solicitada a senha do usuário “pi” em 192.168.0.100. Ao digitar a senha correta, uma mensagem de boas-vindas é apresentada (também conhecida como MOTD ou *message of the day*) e podemos perceber a mudança do *prompt* de comandos (agora pi@raspberrypi:~ \$).

Absolutamente todos os comandos aprendidos aqui (desde que instalados no servidor remoto) são válidos como se estivéssemos sentados na frente do equipamento. Na prática, não faz a menor diferença: no final das contas, o que importa é se possuímos determinada permissão para executar o comando ou não. Ao terminar, podemos digitar “exit” e retornar ao *host* original:

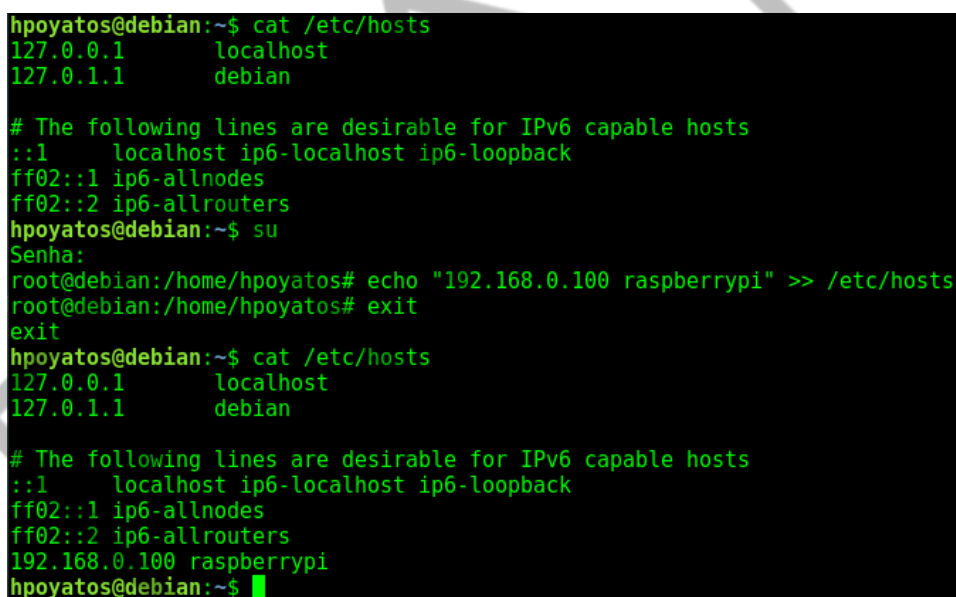
```
hpoyatos@debian:~$ ssh pi@192.168.0.100
The authenticity of host '192.168.0.100 (192.168.0.100)' can't be established.
ECDSA key fingerprint is SHA256:KDxCNmmIVYp7edSLHgoEDE6d61ZuuWg74jfn45+WByw.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.0.100' (ECDSA) to the list of known hosts.
pi@192.168.0.100's password:
Linux raspberrypi 4.9.59-v7+ #1047 SMP Sun Oct 29 12:19:23 GMT 2017 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Feb 17 20:29:39 2018 from 192.168.0.10
pi@raspberrypi:~ $
pi@raspberrypi:~ $
pi@raspberrypi:~ $
pi@raspberrypi:~ $ exit
logout
Connection to 192.168.0.100 closed.
hpoyatos@debian:~$ █
```

Figura 34 – Exemplo de execução do comando SSH ao sair do *host* remoto
Fonte: Elaborado pelo autor (2018)

Você pode tornar o acesso mais prático, definindo o *host* no Linux usando o arquivo `/etc/hosts`; assim, ao se referir, mesmo que o sistema esteja com um servidor DNS (Domain Name System) configurado e traduzindo *hostnames* em IPs, o sistema sempre olha para o `/etc/hosts` primeiro. O exemplo da Figura “Adicionando um host no arquivo `/etc/hosts`” mostra a adição de um novo *host* no arquivo `/etc/hosts`, sem a necessidade de um processador de textos. Para tal, faz-se necessário tornar-se o superusuário `root`, já que o arquivo é protegido. O comando `echo` serve para “ecoar” algo no terminal, ou seja, tudo o que é digitado entre aspas seria repetido no terminal. Os símbolos “>>” (maior-maior) redirecionam a saída padrão do comando `echo` para dentro de um arquivo, incluindo o resultado do comando ao respectivo final. Tome cuidado apenas para não usar um sinal de maior só (“>”), pois o efeito é o mesmo, mas apaga o arquivo `/etc/hosts` inteiro no processo. A execução do comando `cat` mostra o antes e o depois do arquivo:



```
hpoyatos@debian:~$ cat /etc/hosts
127.0.0.1    localhost
127.0.1.1    debian

# The following lines are desirable for IPv6 capable hosts
::1         localhost ip6-localhost ip6-loopback
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
hpoyatos@debian:~$ su
Senha:
root@debian:/home/hpoyatos# echo "192.168.0.100 raspberrypi" >> /etc/hosts
root@debian:/home/hpoyatos# exit
exit
hpoyatos@debian:~$ cat /etc/hosts
127.0.0.1    localhost
127.0.1.1    debian

# The following lines are desirable for IPv6 capable hosts
::1         localhost ip6-localhost ip6-loopback
ff02::1     ip6-allnodes
ff02::2     ip6-allrouters
192.168.0.100 raspberrypi
hpoyatos@debian:~$
```

Figura 35 – Adicionando um host no arquivo `/etc/hosts`
Fonte: Elaborado pelo autor (2018)

A partir de agora, o equipamento remoto também pode ser acessado pelo *hostname* definido em `/etc/hosts` (Figura “Exemplo do comando SSH utilizando *hostname*”):


```
hpoyatos@debian:~$ ssh pi@raspberrypi
The authenticity of host 'raspberrypi (192.168.0.100)' can't be established.
ECDSA key fingerprint is SHA256:KDxCNmmIVYp7edSLHgoEDE6d61ZuuWg74jfn45+WByw.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'raspberrypi' (ECDSA) to the list of known hosts.
pi@raspberrypi's password:
Linux raspberrypi 4.9.59-v7+ #1047 SMP Sun Oct 29 12:19:23 GMT 2017 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Feb 17 20:55:33 2018 from 192.168.0.10
pi@raspberrypi:~$
pi@raspberrypi:~$
pi@raspberrypi:~$
pi@raspberrypi:~$ exit
logout
Connection to raspberrypi closed.
hpoyatos@debian:~$
```

Figura 36 – Exemplo do comando SSH utilizando *hostname*

Fonte: Elaborado pelo autor (2018)

Bem mais simples, não é mesmo? Nada de ficar decorando endereços de IP (se na versão 4 já é ruim, imagine na versão 6).

5.2 SCP – realizando transferências de arquivo seguras

O serviço de **SSH** não possibilita apenas o controle total de um servidor remoto, ele permite, ainda, a transferência de arquivos utilizando esse mesmo canal seguro. O comando **SCP** (*Secure Copy* ou cópia segura) pode ser utilizado para copiar arquivos inteiros de um servidor a outro, sem a necessidade de apelar para o antigo e ultrapassado FTP, realizando o envio de forma sigilosa.

A sintaxe do comando é `scp [origem] [destino]`, referindo-se ao arquivo local da mesma forma que o comando `cp` funciona e o local remoto da maneira que o SSH funciona.

Vamos a um exemplo para simplificar. Na Figura “Exemplo do comando scp transferindo um arquivo local para um servidor remoto”, estando no diretório pessoal do usuário (`/home/hpoyatos`), aponto para o arquivo `Imagem1.png` dentro do subdiretório `Imagens`, copiando-o para o diretório `/tmp` do Raspberry Pi:


```
hpoyatos@debian:~$ scp Imagens/Imagem1.png pi@raspberrypi:/tmp
pi@raspberrypi's password:
Imagem1.png                                100%   0   0.0KB/s   00:00
hpoyatos@debian:~$ █
```

Figura 37 – Exemplo do comando scp transferindo um arquivo local para um servidor remoto
Fonte: Elaborado pelo autor (2018)

Na Figura “Exemplo do comando scp transferindo um arquivo local para um servidor remoto”, repare que podemos realizar o inverso: peço pelo arquivo Imagem1.png, que sei estar no diretório /tmp do Raspberry Pi, e o copio para meu diretório atual (representado pelo símbolo “.”, o ponto):

```
hpoyatos@debian:~$ scp pi@raspberrypi:/tmp/Imagem1.png .
pi@raspberrypi's password:
Imagem1.png                                100%   0   0.0KB/s   00:00
hpoyatos@debian:~$ ls -la Imagem1.png
-rw-r--r-- 1 hpoyatos hpoyatos 0 fev 18 19:14 Imagem1.png
hpoyatos@debian:~$ █
```

Figura 38 – Exemplo do comando scp transferindo um arquivo de um servidor remoto para o local
Fonte: Elaborado pelo autor (2018)

5.3 Subindo um serviço SSH

Até o momento, falamos sobre como acessar um serviço SSH disponível, mas de que maneira podemos configurar um serviço como esse na máquina em que estamos?

Na Figura “Instalação do serviço de SSH”, instalamos o serviço com o comando `$ sudo apt install ssh`:

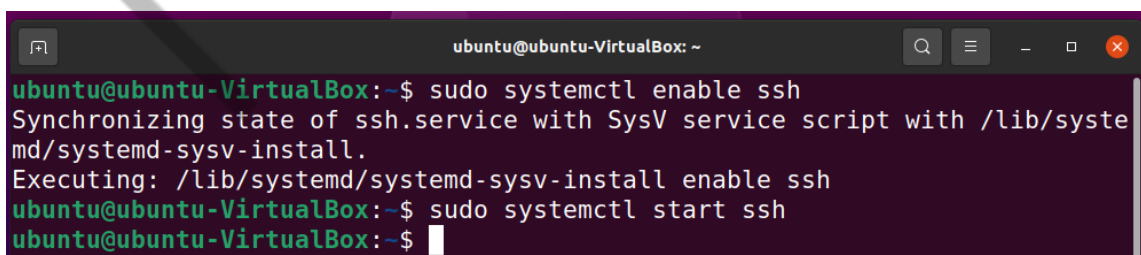
```
hpyatos@debian:~$ sudo apt install ssh
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os seguintes pacotes foram instalados automaticamente e já não são necessários:
  libhidapi-hidraw0 libhidapi-libusb0 linux-image-4.9.0-4-amd64 python-ecdsa
  python-hid python-mnemonic python-pbkdf2 python-protobuf
Utilize 'sudo apt autoremove' para os remover.
Os NOVOS pacotes a seguir serão instalados:
  ssh
0 pacotes atualizados, 1 pacotes novos instalados, 0 a serem removidos e 0 não a
tualizados.
É preciso baixar 189 kB de arquivos.
Depois desta operação, 208 kB adicionais de espaço em disco serão usados.
Obter:1 http://ftp.br.debian.org/debian stretch/main amd64 ssh all 1:7.4p1-10+de
b9u2 [189 kB]
Baixados 189 kB em 1s (181 kB/s)
A seleccionar pacote anteriormente não seleccionado ssh.
(Lendo banco de dados ... 135729 ficheiros e directórios actualmente instalados.
)
A preparar para desempacotar .../ssh_1%3a7.4p1-10+deb9u2_all.deb ...
A descompactar ssh (1:7.4p1-10+deb9u2) ...
Configurando ssh (1:7.4p1-10+deb9u2) ...
```

Figura 39 – Instalação do serviço de ssh
Fonte: Elaborado pelo autor (2018)

Após a instalação, utilizando o comando `systemctl` nos parâmetros `enable` e `start` para, no caso de `enable`, inicializar o serviço sempre que o sistema operacional for inicializado e, no caso de `start`, iniciar o serviço imediatamente:

```
$ sudo systemctl enable ssh
$ sudo systemctl start ssh
```

Comando de prompt 6 – Habilitando no boot e iniciando imediatamente o serviço de ssh
Fonte: Elaborado pelo autor (2019)



```
ubuntu@ubuntu-VirtualBox: ~
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /lib/systemd/syste
md/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl start ssh
ubuntu@ubuntu-VirtualBox:~$
```

Figura 40 – Subindo o serviço de SSH
Fonte: Elaborado pelo autor (2021)

5.4 Tornando o SSH ainda mais seguro

Embora seja um serviço que conte com criptografia e possua uma configuração inicial razoavelmente segura, o acesso remoto provido pelo SSH é muito visado por eventuais invasores, justamente por seu potencial de manipulação remota (embora não seja, nem de longe, a única maneira de invadir uma máquina).

Com o intuito de proteger ainda mais o serviço, seguem algumas dicas rápidas sobre como tornar o SSH mais seguro do que ele já é:

- **Use senhas seguras em seus usuários:** o ataque mais banal que se procura fazer é um ataque de força bruta (ou *brute force attack*), no qual se conecta ao serviço e tenta-se adivinhar a senha, testando todas as combinações possíveis. Sem muita dificuldade, o atacante pode automatizar o processo, usando todas as palavras do vocabulário de um idioma, conhecido como ataque de dicionário, ou seja, senhas muito simples são facilmente descobertas e garantem um acesso simples e rápido. Não deixe isso acontecer.
- **Configure o serviço:** por falar em força bruta, o arquivo `/etc/ssh/sshd_config` permite uma série de parametrizações interessantes, como determinar o número de tentativas máximas para senhas. Garanta que o usuário *root* não possa se autenticar e restrinja o serviço para as interfaces de rede em que ele precise realmente ficar disponível. Você pode restringir o acesso apenas para *hosts* conhecidos (aqueles dos quais você possui as chaves públicas). Não será prático adicionar novos *hosts*, mas torna-se bem mais seguro. Aprenderemos, na sequência, sobre o editor de textos vi e como alterar arquivos com ele.
- **Restrinja os usuários que podem usar SSH:** nem todo mundo precisa ter acesso remoto. Para fazê-lo, adicione, no arquivo `/etc/ssh/sshd_config`, as diretivas ***allowuser***, para especificar os usuários permitidos (separe-os por vírgula), ou ***allowgroup***, para liberar grupos. As diretivas ***denyuser*** e ***denygroup*** permitem definir quais usuários e grupos estão definitivamente proibidos de usá-lo.

- **Troque a porta de serviço:** no mesmo arquivo `/etc/ssh/sshd_config`, você pode trocar o número da porta de serviço por outro número que não seja o óbvio 22.
- **Apenas IPs permitidos:** aprenderemos sobre configuração de *firewall* em um capítulo de uma etapa seguinte; quando acontecer, você pode usar o comando **iptables** para definir quais IPs podem acessar a porta de serviço na qual o SSH está.
- **Identifique-se primeiro:** outro truque muito interessante é a possibilidade de implementar um Port Knocking, que em tradução livre significa “batidas na porta”; configuramos o firewall para liberar a porta de serviço apenas quando pacotes de dados são enviados para portas específicas em uma determinada sequência; podemos até mesmo configurar a sequência para fechar a porta ao sair! Muito seguro, não é?

6 VI: O MELHOR EDITOR DE TEXTO

O título desta seção é tendencioso e está longe de ser imparcial, mas é, também, uma verdade: embora existam outros editores de texto, como o **emacs** ou o **nano** (que é bem fácil de usar), o vi (lê-se vi-ai) ou vim (**Vi Improved** ou “vi melhorado”) possui uma infinidade de recursos interessantes e poderosos.

Praticamente todas as letras minúsculas e maiúsculas disparam comandos no editor; portanto, para que este tutorial não fique muito complexo (e chato), vou me restringir às funcionalidades mais comuns e úteis!

6.1 Entrando e saindo do vi

Para entrar no vi, basta digitar, no terminal, `$ vi`. Normalmente, o comando é acompanhado com o nome do arquivo que se deseja criar ou editar, conforme podemos ver na Linha de comando “Exemplo do comando vi criando ou editando arquivo.txt”, a seguir:

```
$ vi arquivo.txt
```

Comando de prompt 7 – Exemplo do comando vi criando ou editando arquivo.txt
Fonte: Elaborado pelo autor (2018)

A tela do editor é muito simples, como podemos ver na Figura “Tela inicial do vi criando arquivo.txt”:

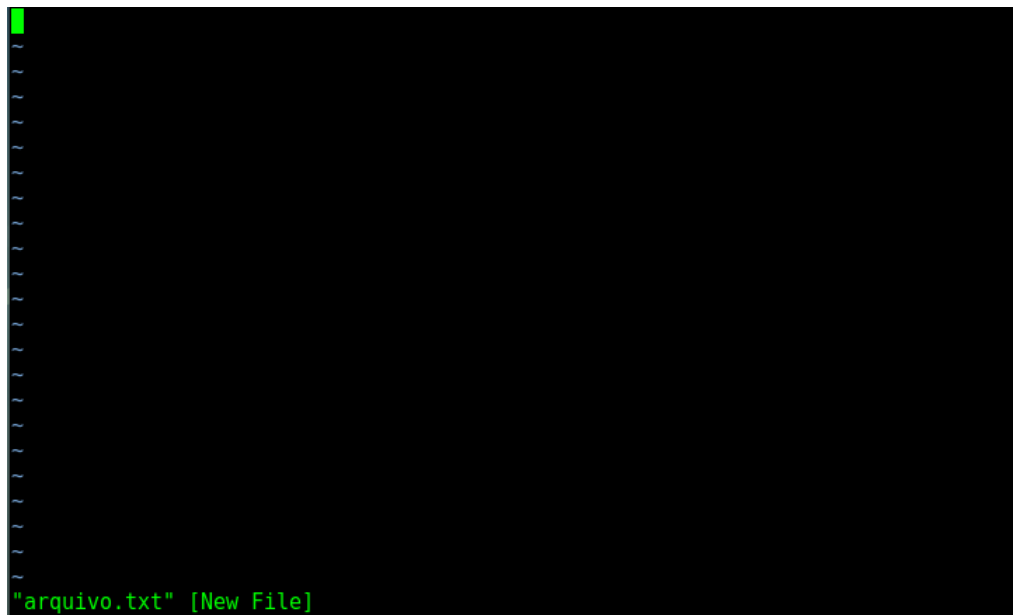


Figura 41 – Tela inicial do vi criando arquivo.txt
Fonte: Elaborado pelo autor (2018)

Para sair do editor, você deve digitar: “:q” (dois-pontos e a letra q minúscula, de *quit*). Sugiro acostumar-se com a combinação das teclas [ESC], [:], [q] e [ENTER], pois você pode estar em modo de edição de texto, e a tecla [ESC] fará voltar ao modo de comandos.

6.2 Os modos de comando e de edição de texto

O vi conta com dois modos: um modo de comandos, no qual as teclas que você digitar dispararão comandos no editor, e o modo de edição de texto, no qual tudo aquilo que você digitar fará, literalmente, parte do documento.

Existem várias maneiras de migrar do modo de comando para o modo de edição de texto, vou me restringir às quatro mais interessantes:

1) A mais simples é a tecla [i], por meio da qual entramos em modo de edição de texto **no ponto em que o cursor estiver no momento**. Na Figura “Editando um arquivo com vi usando [i]”, digite [i] e a frase “Vi é muito legal!”, seguidos de [ESC], para voltar ao modo de comandos:



Figura 42 – Editando um arquivo com vi usando [i]
Fonte: Elaborado pelo autor (2018)

2) Outra forma é a tecla [a], por meio da qual entramos em modo de edição de texto **uma posição após o ponto em que o cursor estiver no momento**. Na Figura “Editando um arquivo com vi usando [a]”, digite [a] com o cursor posicionado em “!” e, dessa maneira, podemos continuar a frase exatamente de onde havia parado; digite “E muito rápido, também.”, seguido de [ESC], para voltar ao modo de comandos:

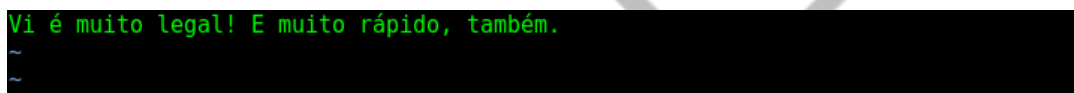


Figura 43 – Editando um arquivo com vi usando [a]
Fonte: Elaborado pelo autor (2018)

3) O terceiro modo é editar com [A], e entramos em modo de edição **no final da frase em que estamos**. Para a demonstração da Figura “Editando um arquivo com vi usando [A]”, retorne com [←] até chegar ao início da frase, embora [^] nos posicione imediatamente no início da frase. Ao digitar [A], somos enviados imediatamente ao final da frase, em modo de edição, e digite “E cheio de truques!”, seguido de [ESC], para voltar ao modo de comandos:

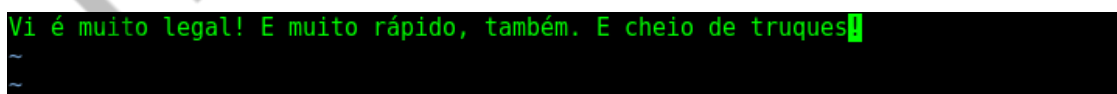
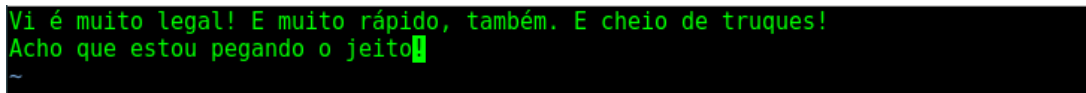


Figura 44 – Editando um arquivo com vi usando [A]
Fonte: Elaborado pelo autor (2018)

4) Por último, podemos usar [o] e entrar no modo de edição **na linha seguinte àquela em que estamos**. Para a demonstração da Figura “Editando um arquivo com vi usando [o]”, basta digitar [o] e a frase “Acho que estou pegando o jeito!”, seguidos de [ESC], para voltar ao modo de comandos:



```
Vi é muito legal! E muito rápido, também. E cheio de truques!  
Acho que estou pegando o jeito!
```

Figura 45 – Editando um arquivo com vi usando [o]
Fonte: Elaborado pelo autor (2018)

E acredite, existem ainda outras maneiras. Quanto a retornar ao modo de comandos, acho que você já entendeu como funciona.

6.3 Salvando o arquivo

Para salvar o arquivo, você deve estar em modo de comandos, e pode utilizar as teclas [:], [w] (de *write*) e [ENTER]. Para salvar e sair do arquivo, você pode combinar as letras “w” e “q”, usando a sequência [:], [w], [q] e [ENTER] (:wq), embora eu prefira [:], [x] e [ENTER], que faz o serviço com uma tecla a menos.

Quer sair sem salvar? Utilize [:], [q], [!] e [ENTER] (ou :q!) em modo de comando.

6.4 Numeração de linhas e como ir direto para uma delas

Para os demais exemplos, vamos aumentar o tamanho do arquivo, como podemos ver na Figura “Exemplo de documento no vi”:


```
Vi é muito legal! E muito rápido, também. E cheio de truques!  
Acho que estou pegando o jeito!  
  
Ele parece complicado mas nada que uma boa prática não resolva!  
  
Por ser muito rápido e leve, é ideal para editar arquivos em linha de comando.  
Mas, se eu quisesse alguém rápido e leve, escolheria o nano! Além destas  
duas características, o vi é extremamente poderoso, e neste tutorial  
você verá apenas um lampejo disso!  
  
Quem sabe você não toma gosto e se aprofunda depois, descobrindo mais e mais  
atalhos?  
  
Talvez um dia alguém lhe fale do emacs, mas o vi é mais legal, vai por mim!  
(E o prof. Poyatos continua alimentando um dos Flame Wars ou "Fla-Flus" mais  
antigos da tecnologia... vi x emacs!)  
~  
~  
~  
~  
~  
"arquivo.txt" 16 lines, 713 characters
```

Figura 46 – Exemplo de documento no vi
Fonte: Elaborado pelo autor (2018)

Segue o texto no *box*, caso queira me acompanhar:

```
Vi é muito legal! E muito rápido, também. E cheio de truques!  
Acho que estou pegando o jeito!  
  
Ele parece complicado, mas nada que uma boa prática não resolva!  
  
Por ser muito rápido e leve, é ideal para editar arquivos em linha de comando.  
Mas, se eu quisesse algo rápido e leve, escolheria o nano! Além dessas  
duas características, o vi é extremamente poderoso e, neste tutorial,  
você verá apenas um lampejo disso!  
  
Quem sabe você não toma gosto e se aprofunda depois, descobrindo mais e mais atalhos?  
  
Talvez, um dia, alguém lhe fale do emacs, mas o vi é mais legal, vai por mim!  
(E o prof. Poyatos continua alimentando um dos Flame Wars ou "Fla-Flus" mais  
antigos da tecnologia... vi x emacs!)
```

Não ver a numeração das linhas pode ser um problema em algumas situações. Imagine no caso de editar um código-fonte cujo depurador apontou um erro na linha 11? Para habilitar as linhas, digite a sequência [:], "set nu" e [ENTER] (ou :set nu), no modo de comandos:

```
1 Vi é muito legal! E muito rápido, também. E cheio de truques!
2 Acho que estou pegando o jeito!
3
4 Ele parece complicado mas nada que uma boa prática não resolva!
5
6 Por ser muito rápido e leve, é ideal para editar arquivos em linha de comando.
7 Mas, se eu quisesse alguém rápido e leve, escolheria o nano! Além destas
8 duas características, o vi é extremamente poderoso, e neste tutorial
9 você verá apenas um lampejo disso!
10
11 Quem sabe você não toma gosto e se aprofunda depois, descobrindo mais e mais
12 atalhos?
13
14 Talvez um dia alguém lhe fale do emacs, mas o vi é mais legal, vai por mim!
15 (E o prof. Poyatos continua alimentando um dos Flame Wars ou "Fla-Flus" mais
16 antigos da tecnologia... vi x emacs!)
~
~
~
```

Figura 47 – Exemplo de documento no vi com numeração de linhas
Fonte: Elaborado pelo autor (2018)

Se quisermos nos dirigir rapidamente à linha 11, podemos usar a tecla [↓] dez vezes até chegar lá. Podemos usar as setas a fim de navegar para qualquer parte do documento, **desde que estejamos em modo de comando**: usar as setas em modo de edição sujará seu documento com os caracteres mais bizarros possíveis e, se meu aviso veio tarde demais, a seção **6.6 Apagando textos** é para você.

Ir até a linha 11 é fácil, mas e se fosse a linha 2853? Melhor aprender a ir direto para onde precisa, você não acha? Experimente digitar [:], "11" e [ENTER]:

```
1 Vi é muito legal! E muito rápido, também. E cheio de truques!
2 Acho que estou pegando o jeito!
3
4 Ele parece complicado mas nada que uma boa prática não resolva!
5
6 Por ser muito rápido e leve, é ideal para editar arquivos em linha de comando.
7 Mas, se eu quisesse alguém rápido e leve, escolheria o nano! Além destas
8 duas características, o vi é extremamente poderoso, e neste tutorial
9 você verá apenas um lampejo disso!
10
11 Quem sabe você não toma gosto e se aprofunda depois, descobrindo mais e mais
12 atalhos?
13
14 Talvez um dia alguém lhe fale do emacs, mas o vi é mais legal, vai por mim!
15 (E o prof. Poyatos continua alimentando um dos Flame Wars ou "Fla-Flus" mais
16 antigos da tecnologia... vi x emacs!)
~
~
~
```

Figura 48 – Exemplo para acessar uma linha específica no vi
Fonte: Elaborado pelo autor (2018)

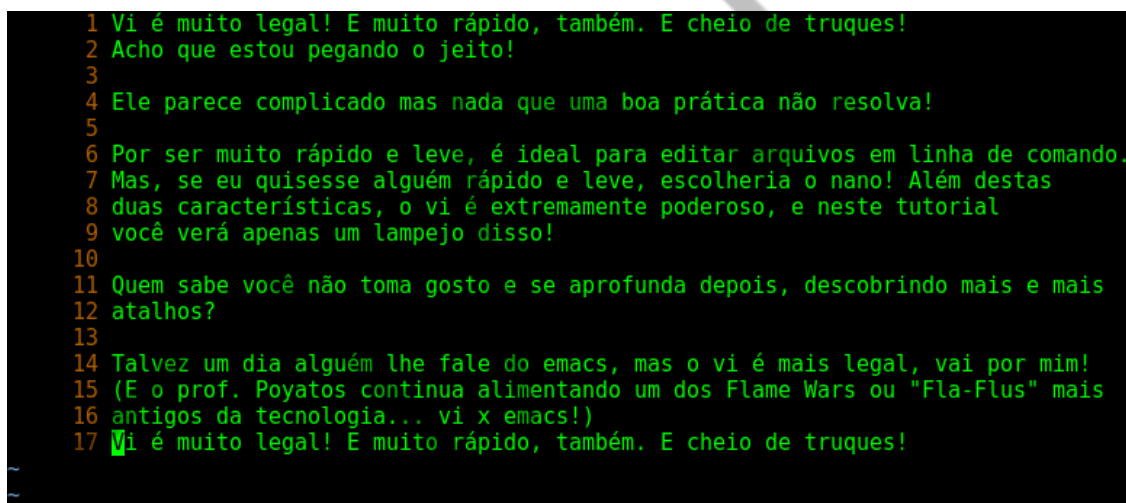
Não negue, você está sorrindo neste exato momento.

6.5 Copiando e colando textos

Não seríamos ninguém sem a possibilidade de copiar e colar textos. Você não usaria o **vi** se isso não fosse possível (bem, nem eu), e, talvez, muitos falem mal do **vi** justamente por não saberem como fazer.

Vamos começar copiando uma linha inteira. Vá para a primeira linha (:1 em modo de comandos resolve) e digite [y][y] (de *yank* ou “arrancar”; por quê? Não sei) no modo de comando. Embora não tenha avisado, acredite, ele copiou a primeira linha.

Vá, agora, para a última linha do arquivo (experiente [G] em modo de comando) e digite [p] (de *paste* ou colar):



```
1 Vi é muito legal! E muito rápido, também. E cheio de truques!
2 Acho que estou pegando o jeito!
3
4 Ele parece complicado mas nada que uma boa prática não resolva!
5
6 Por ser muito rápido e leve, é ideal para editar arquivos em linha de comando.
7 Mas, se eu quisesse alguém rápido e leve, escolheria o nano! Além destas
8 duas características, o vi é extremamente poderoso, e neste tutorial
9 você verá apenas um lampejo disso!
10
11 Quem sabe você não toma gosto e se aprofunda depois, descobrindo mais e mais
12 atalhos?
13
14 Talvez um dia alguém lhe fale do emacs, mas o vi é mais legal, vai por mim!
15 (E o prof. Poyatos continua alimentando um dos Flame Wars ou "Fla-Flus" mais
16 antigos da tecnologia... vi x emacs!)
17 Vi é muito legal! E muito rápido, também. E cheio de truques!
```

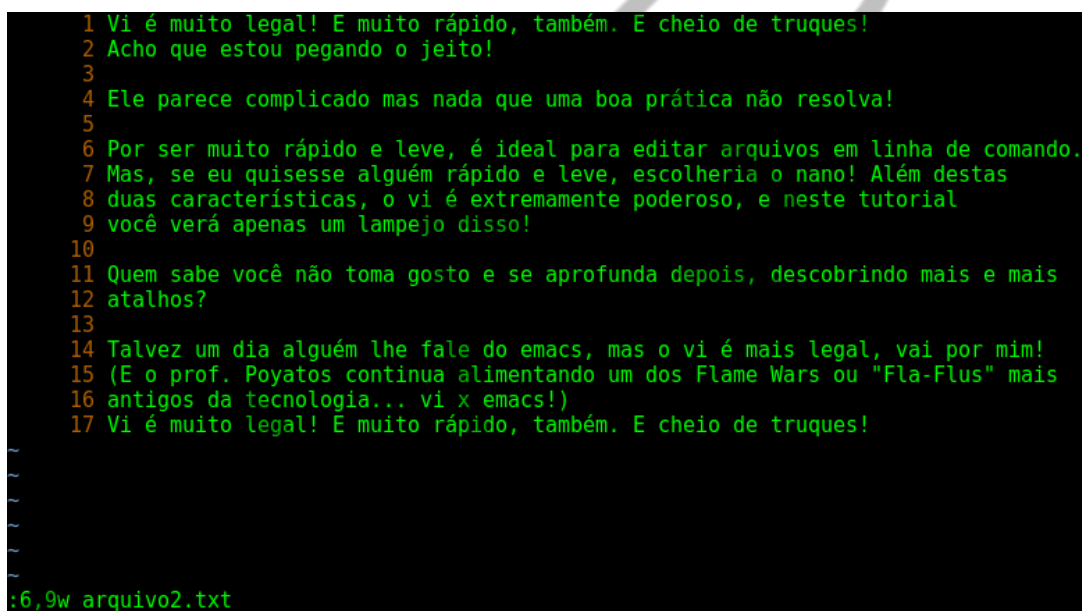
Figura 49 – Exemplo de como copiar e colar uma linha no vi
Fonte: Elaborado pelo autor (2018)

A propósito, um único [y] também copiaria, mas ele copia um parágrafo (ou melhor, até achar uma linha em branco); ou seja, na primeira linha, ele copiaria duas linhas, enquanto, com o cursor na linha 6, teria copiado quatro.

Para copiar um número específico de linhas, basta colocar a quantidade antes do comando, ou seja, digitar 10yy em modo de comando fará o vi copiar dez linhas, sendo a linha em que você está e as nove seguintes.

Você pode copiar trechos de linha também. As teclas `yw` (de *word* ou palavra) copiarão uma única palavra (ou seja, até o `vi` achar um *caractere* de espaço), `y$` copiará do ponto em que o cursor esteja até o final da linha, enquanto `y^` faz o contrário, copiará de onde estivermos até o início da linha. Talvez você conheça **expressões regulares** e tenha reconhecido o “^” para início de sentença e o “\$” para final de sentença; caso contrário, recomendo, é sempre muito útil!

E se quisermos copiar um trecho do documento para outro arquivo? Na Figura “Exemplo de como copiar um trecho do arquivo para outro”, podemos ver como copiar das linhas 6 a 9 de um arquivo para outro, que chamei de `arquivo13.txt`:



```
1 Vi é muito legal! É muito rápido, também. É cheio de truques!
2 Acho que estou pegando o jeito!
3
4 Ele parece complicado mas nada que uma boa prática não resolva!
5
6 Por ser muito rápido e leve, é ideal para editar arquivos em linha de comando.
7 Mas, se eu quisesse alguém rápido e leve, escolheria o nano! Além destas
8 duas características, o vi é extremamente poderoso, e neste tutorial
9 você verá apenas um lampejo disso!
10
11 Quem sabe você não toma gosto e se aprofunda depois, descobrindo mais e mais
12 atalhos?
13
14 Talvez um dia alguém lhe fale do emacs, mas o vi é mais legal, vai por mim!
15 (E o prof. Poyatos continua alimentando um dos Flame Wars ou "Fla-Flus" mais
16 antigos da tecnologia... vi x emacs!)
17 Vi é muito legal! É muito rápido, também. É cheio de truques!

:6,9w arquivo2.txt
```

Figura 50 – Exemplo de como copiar um trecho do arquivo para outro
Fonte: Elaborado pelo autor (2018)

6.6 Apagando textos

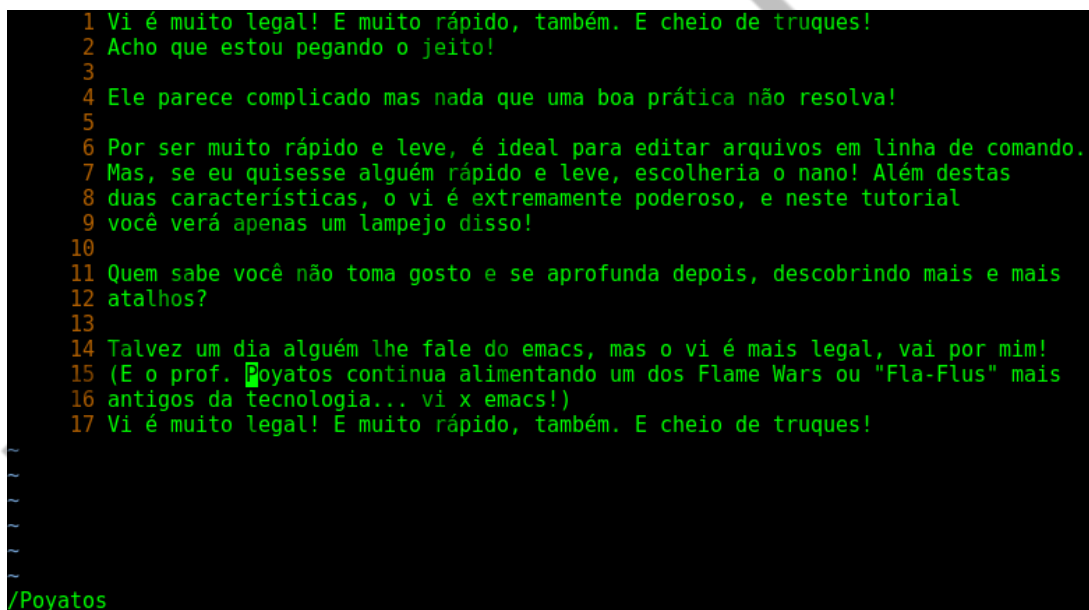
Para apagar textos, os procedimentos são os mesmos usados para copiar, mas você substitui o “y” por “d” (de *delete* ou apagar); ou seja, digitar `dd` em modo comando apaga uma única linha, enquanto `10dd` apaga a linha em que você está e mais nove; `dw` apaga a palavra em que você está, `d$` apaga do ponto em que você está até o final da linha, enquanto `d^` apaga de onde você está até o início. Quer apagar um único caractere? `d→`.

Apagou algo errado? Digite [u] e [ENTER] (de *undo* ou desfazer); com [u] você pode desfazer o último comando, e **APENAS O ÚLTIMO**. Se for o penúltimo, danou-se.

Quer recortar? Use os comandos “d” também. Talvez você se pergunte: “ué, dd não é para apagar uma linha?”. Na verdade, ele recorta a linha. Se você usar o [p] (de *paste*) na sequência, ele realiza um recortar-e-colar. Engenhoso, não?

6.7 Localizando e substituindo palavras

É indispensável saber realizar uma busca por palavra-chave em um documento. Para fazer isso no vi, digite [/], a palavra que você deseja e [ENTER], como você pode ver na Figura “Exemplo de busca por palavra-chave no vi”:



```
1 Vi é muito legal! É muito rápido, também. É cheio de truques!
2 Acho que estou pegando o jeito!
3
4 Ele parece complicado mas nada que uma boa prática não resolva!
5
6 Por ser muito rápido e leve, é ideal para editar arquivos em linha de comando.
7 Mas, se eu quisesse alguém rápido e leve, escolheria o nano! Além destas
8 duas características, o vi é extremamente poderoso, e neste tutorial
9 você verá apenas um lampejo disso!
10
11 Quem sabe você não toma gosto e se aprofunda depois, descobrindo mais e mais
12 atalhos?
13
14 Talvez um dia alguém lhe fale do emacs, mas o vi é mais legal, vai por mim!
15 (E o prof. Poyatos continua alimentando um dos Flame Wars ou "Fla-Flus" mais
16 antigos da tecnologia... vi x emacs!)
17 Vi é muito legal! É muito rápido, também. É cheio de truques!

~/Poyatos
```

Figura 51 – Exemplo de busca por palavra-chave no vi
Fonte: Elaborado pelo autor (2018)

Se for uma palavra-chave por mais de uma ocorrência (neste exemplo, a melhor palavra é justamente “mais”), digite [n] para pular para a próxima ocorrência; se quiser voltar para a anterior, use [N].

E para substituir termos no documento? Na Linha de comando “Exemplo de substituição da palavra ‘mais’ por ‘menos’ no vi”, mostro como substituir todas as ocorrências da palavra “mais” por “menos”:

```
:%s/mais/menos/gc
```

Comando de prompt 8 – Exemplo de substituição da palavra “mais” por “menos” no vi
Fonte: Elaborado pelo autor (2018)

O “c” ao final do comando é para pedir confirmação para cada substituição; se você remover o “c” do comando, ele troca todas as ocorrências sem a menor cerimônia. A letra “g” antes dela indica que a substituição é global, ou seja, para todas as ocorrências; sua ausência faria o vi substituir apenas a primeira palavra “mais” que encontrasse, deixando as demais intactas. Veja como fica a execução da linha de comando “ exemplo de substituição da palavra “mais” por “menos” no vi” na figura “Exemplo de substituição de termos no vi”:

```
1 Vi é muito legal! E muito rápido, também. E cheio de truques!
2 Acho que estou pegando o jeito!
3
4 Ele parece complicado mas nada que uma boa prática não resolva!
5
6 Por ser muito rápido e leve, é ideal para editar arquivos em linha de comando.
7 Mas, se eu quisesse alguém rápido e leve, escolheria o nano! Além destas
8 duas características, o vi é extremamente poderoso, e neste tutorial
9 você verá apenas um lampejo disso!
10
11 Quem sabe você não toma gosto e se aprofunda depois, descobrindo menos e menos
12 atalhos?
13
14 Talvez um dia alguém lhe fale do emacs, mas o vi é menos legal, vai por mim!
15 (E o prof. Poyatos continua alimentando um dos Flame Wars ou "Fla-Flus" menos
16 antigos da tecnologia... vi x emacs!)
17 Vi é muito legal! E muito rápido, também. E cheio de truques!

4 substitutions on 3 lines
```

Figura 52 – Exemplo de substituição de termos no vi
Fonte: Elaborado pelo autor (2018)

Agora você entendeu a ideia!

CONCLUSÃO

Neste capítulo, abordamos alguns pontos muito importantes no sistema do pinguim. Começamos pelo gerenciamento de processos, indispensável na administração de sistemas, e pudemos perceber o quanto o Linux nos permite gerenciá-los facilmente, disponibilizando diversas ferramentas para tal.

Passamos pelo serviço mais importante do sistema operacional, o ssh. Considero impensável sobreviver sem ele nos dias de hoje. Como configurar um servidor em *cloud* sem ele? Absolutamente impossível.

Por fim, seríamos incapazes de editar arquivos de configuração sem a ajuda do melhor editor de textos em linha de comando do mercado, o vi. A aliança ssh + vi + os comandos que você aprendeu até agora, o céu é o limite.

EXEMPLO

REFERÊNCIA

FERREIRA, R. E. **Linux** – Guia do Administrador do Sistema. 14. ed. São Paulo: Novatec, 2008.

EMANIP

GLOSSÁRIO

SSH	<i>Secure Shell</i> , ou <i>shell</i> seguro. Trata-se de um serviço de acesso remoto que permite realizar comandos em outra máquina (como o antigo <i>telnet</i>), mas que trafega as interações em um túnel criptografado.
------------	---